Supplementary Material for: Proof-Valid Serving and Benchmarking for Approximate LLM Caches under Premise Erasure

Jianfeng Xu
Koguan School of Law, China Institute for Smart Justice,
School of Computer Science, Shanghai Jiao Tong University,
Shanghai 200030, China
Corresponding author: xujf@sjtu.edu.cn

Online Resource 1, *Knowledge and Information Systems*

This supplement reports extended controlled audits, descriptive analyses, case-level results, and implementation checks. The main text presents the core contracts, workflow, evaluation design, and headline results; this document supplies detailed supporting analyses and retained-output mappings. Throughout, PVCacheServe refers to the proof-valid cache serving and benchmarking workflow defined in the main text.

Local section, table, figure, equation, and algorithm numbers carry an "S" prefix (e.g. Section S4, Table S4.1, Algorithm S1.1). References to the main manuscript are marked "main-text" and use its unprefixed numbers and appendix letters. Each document has its own bibliography.

Terminology. A support schema declares dependency rules. A certificate is an available witness under those rules at a named audit state. A stored certificate record or envelope binds the response to its declared support and issuance metadata; retaining that record does not establish availability at a later state.

S1 Full Controlled-Availability Audits

This section specifies the evaluation contracts, calibration procedures, and measurement conventions; the accompanying scripts record the remaining implementation-level configuration.

Shared configuration. Unless otherwise stated, simulations use seed 42 and erasure grid $\{0, 0.05, 0.1, 0.15, 0.2, 0.3, 0.4, 0.5\}$. The coded-recovery failure target and the trace foreign-centroid score-exceedance target both use $\delta = 0.02$, but constrain different events. Throughout this supplement, round denotes nearest-integer rounding with ties to even. Fault-sweep means are displayed to one decimal place using decimal half-up rounding from integer totals.

Policies and metrics. The trace audit compares static-threshold, globally calibrated adaptive-threshold, and certificate-gated benchmark policies. These are policy models, not executions of GPTCache or vCache [1, 2]. The adaptive threshold is calibrated in-sample to a request-level foreign-centroid score-exceedance target of $\delta = 0.02$. It is neither vCache's per-prompt threshold learner nor a held-out or served-conditional answer-error guarantee. At each erasure rate, all policies use the same request-level erasure realization. Table S1.1 gives the thresholds, and Table S1.2 reports selected operating points with served requests as the error denominator.

Error denominators. For a policy serving $N_{\text{serve}} > 0$ requests, let N_{false} and N_{premise} be the two disjoint error counts. The served-conditional error rate is

$$e_{\text{serve}} = \frac{N_{\text{false}} + N_{\text{premise}}}{N_{\text{serve}}},$$

and the corresponding unconditional request-level error mass is

$$e_{\text{request}} = \frac{N_{\text{false}} + N_{\text{premise}}}{N} = \frac{N_{\text{serve}}}{N}\, e_{\text{serve}}.$$

If no requests are served, the served-conditional rate is undefined rather than zero. Table S1.2 uses e_{serve}; the operating curves in Section S1.8 use e_{request}.

Availability estimator. Let o_i indicate certificate survival for request i, and let $p_i = (1 - \varepsilon)^{n_{C(i)}}$ with $n_{C(i)} = \widehat{\kappa}_{C(i)} - x_{C(i)}$. The all-request availability audit compares $\widehat{R} = \frac{1}{N}\sum_i o_i$ with $\overline{R} = \frac{1}{N}\sum_i p_i$. Under independent query-local draws, $\mathrm{SE}(\widehat{R}) = \sqrt{\sum_i p_i(1-p_i)}/N$. This estimator concerns all requests, not only threshold-selected serves; for a nonempty fixed own-hit set H, conditional availability uses $\sum_{i\in H} o_i/|H|$, with prediction $\sum_{i\in H} p_i/|H|$. The recorded maximum absolute simulation–prediction differences are 0.00502 for SearchQueries and 0.00634 for LmArena over the reported grid. At $N = 20{,}000$, the standard error at any one rate is bounded by $1/(2\sqrt{N}) \approx 0.00354$. Equality requires $p_i = 1/2$ for every request; the point-specific standard error instead depends on the full set of probabilities $\{p_i\}$. Because the grid points reuse one random stream (Section S1.6), these differences are not independent across the grid, and the recorded maxima are descriptive agreement checks, not significance tests or simultaneous confidence bounds.

S1.1 Evidence footprints and independent erasure

We use the HotpotQA distractor validation split (7,405 questions) and the MuSiQue development split (2,417 answerable questions) with their annotated evidence sets [3, 4]. For HotpotQA, E_q contains annotated supporting sentences; for MuSiQue, it contains annotated supporting paragraphs. For availability and designated-payload calculations, we use the annotation-defined AND support specification $P_q^* = E_q$, with $e_q = |E_q|$. The audit measures survival of these designated sets; it does not verify a model-generated response or establish generation provenance.

For HotpotQA, we embed questions using all-MiniLM-L6-v2 [5] and form an undirected graph joining question pairs whose cosine similarity is at least $\tau_c = 0.75$. Its connected components provide the grouping labels. Evidence counts concentrate at $e_q = 2$, with mean 2.43, whereas the 7,331 embedding components are predominantly singletons. The descriptive question-level Spearman association is near zero ($\rho = 0.003$, nominal $p = 0.798$; main-text Figure 3(a,b)). These statistics characterize this grouping and annotated workload; they do not establish a general relationship between semantic equivalence and evidence demand.

For the HotpotQA audit, independent evidence erasure gives

$$R(\varepsilon) = \frac{1}{N}\sum_q (1-\varepsilon)^{e_q}, \qquad N = 7{,}405.$$

Means from 20 Monte Carlo repetitions differ from this prediction by less than 0.001 on the reported grid. Replacing evidence counts by embedding-component sizes changes predicted availability by up to 0.292, with mean absolute count difference 1.415 (main-text Figure 3(c)). Component size is therefore an inadequate evidence-footprint proxy in this audit.

Main-text Figure 3(a,c) also includes class-size references from two 20,000-prompt vCache benchmarks [2, 6]. Those curves use full class sizes as synthetic exponents. The policy overlay in

Sections S1.3–S1.6 instead protects $\lceil |C|/2 \rceil$ units per class and uses the residual count $\lfloor |C|/2 \rfloor$. Neither construction estimates actual generation provenance. Section S1.9 separately perturbs HotpotQA evidence counts.

S1.2 Instrumented retrieval and dependent failures

Instrumented retrieval records. The corpus contains 66,581 title-deduplicated HotpotQA context records, retaining the first passage encountered for each title. Questions and records use the same sentence encoder [5]. At retrieval depth $k = 10$, the constructor Π of main-text Definition 3.4 builds the premise-leaf records of main-text Definition 3.1, recording source identities, payload hashes, and fixed audit metadata for two sets: the *gold-in-retrieval* set (retrieved records whose titles occur in the question's gold supporting-title set) and the *full-context* set (all retrieved records). Title agreement does not ensure coverage of every annotated supporting sentence. Each audited set is checked as a conjunction of its record-availability predicates; this checks set survival rather than answer-level certification. The full-context conjunction requires every recorded context item to remain available; it does not assert that every item was necessary for the response. Section S7.7 specifies record construction.

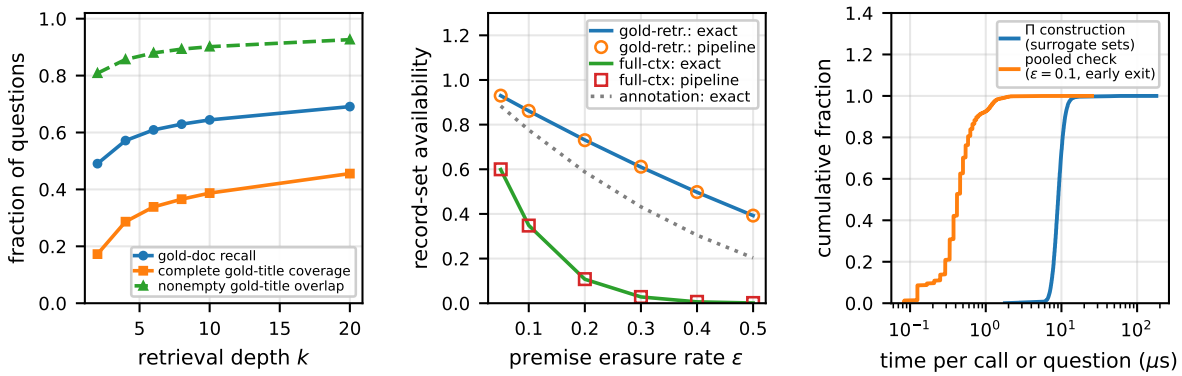
At $k = 10$, 90.2% of questions retrieve at least one gold title, and 38.7% retrieve all gold titles; mean gold-document recall is 64.4% (Figure S1.1(a)). The mean number of retrieved gold documents is 1.29, or approximately 1.43 conditional on nonempty overlap. Missing gold titles are retrieval coverage failures, distinct from subsequent record erasure.

Independent record-erasure simulations show a maximum deviation of approximately 0.0016 from predictions for gold-in-retrieval and 0.0012 for full-context (Figure S1.1(b)). The former conditions on 6,676 questions with nonempty gold-title overlap; the latter includes all 7,405 questions.

The experiment uses erasure masks and a fixed local hash registry; version, policy, and index state remain unchanged. A separate memory-resident dataset-context microbenchmark reports median record-construction cost 9.0 μs per question and pooled checker-call medians 0.67 μs for intact sets and 0.42 μs under $\varepsilon = 0.1$ with early exit (Figure S1.1(c)). Section S7.7 specifies the timing workload and measurement exclusions.

Group failures and deletion bursts. The annotations provide the grouping needed to test the independent-leaf model. We compare independent leaf erasures with independent group failures and fixed-size group-deletion bursts, using supporting documents as groups in HotpotQA and supporting paragraphs as groups in MuSiQue. If a question has e_q evidence leaves in g_q supporting groups, independent group failures give availability $(1 - \varepsilon)^{g_q}$. For HotpotQA, each supporting document groups one or more evidence leaves, so $g_q \leq e_q$ and group-failure availability is at least leaf-i.i.d. availability for every question. Mean counts are 2.00 documents and 2.43 evidence leaves. The simulated group law differs from its prediction by at most approximately 0.0017. At $\varepsilon = 0.3$, the exact workload-average group–leaf availability gap is approximately 0.058 (main-text Figure 4(b)). For MuSiQue, each supporting paragraph is one evidence unit, so $g_q = e_q$ and the two predictions coincide.

For the burst arm, a burst fires with per-question probability $p_b(q) = \min\{\varepsilon/f_q, 1\}$ and deletes a uniformly selected subset of $a_q = \max\{1, \lfloor g_q/2 \rfloor\}$ of the question's g_q supporting groups, $f_q = a_q/g_q$ being the fraction one burst removes. Each group then has erasure marginal $p_b(q)f_q = \min\{\varepsilon, f_q\}$: the marginals match the nominal rate when $\varepsilon \leq f_q$, and cap-active cases must be interpreted separately. An unprotected nonempty certificate survives exactly when no burst fires, giving $R_{\text{burst}}(q) = 1 - p_b(q)$. On HotpotQA the marginal is matched throughout the grid, and at $\varepsilon = 0.5$ every burst fires: availability is zero, compared with approximately 0.203 under leaf-i.i.d. erasure and 0.25 under independent document failures (main-text Figure 4(a)). For MuSiQue, cap-active three-group questions reduce the mean



(a) Retrieval coverage versus depth.

(b) Record availability.

(c) Local record construction and checking costs.

Figure S1.1: Retrieval coverage, record availability, and separate local timings. (a) Gold-title recall, complete coverage, and nonempty overlap. (b) Independent-erasure predictions and 20-repetition checks; gold-in-retrieval conditions on nonempty gold-title overlap. (c) Dataset-context surrogate timings, with checking at $\varepsilon = 0.1$ and early exit.

achieved marginal $\frac{1}{N}\sum_q \min\{\varepsilon, f_q\}$ to 0.448 at nominal $\varepsilon = 0.5$ (main-text Figure 4(c)). The matched-marginal arms show that marginal erasure rates alone do not determine conjunctive availability: under an arbitrary joint failure law, the relevant quantity is $\Pr[D_{\exp}(q, S) \subseteq \widetilde{B}]$, not a product of marginals. MuSiQue's cap-active burst points additionally change the achieved marginal and are not matched-rate comparisons. All three arms use query-local draws and estimate mean per-query availability; they do not estimate joint workload reliability under one shared corpus-erasure realization.

S1.3 Synthetic trace overlay

We use the first 20,000 rows of each released vCache benchmark, SemBenchmarkSearchQueries and SemBenchmarkLmArena [2, 6]. They contain 6,113 and 3,372 `id_set` classes, respectively. The archived configurations identify `response_llama_3_8b` for SearchQueries and `response_gpt-4o-mini` for LmArena. The audit uses class labels and embeddings, not response text.

Because these datasets contain no premise provenance, their class sizes supply synthetic evidence footprints rather than estimates of actual evidence demand. Sections S1.4 and S1.6 define the overlay scores and policy-accounting procedure. Main-text Proposition A.1 bounds availability differences caused by exposed-count misspecification; it does not validate class size as an evidence-demand estimator.

S1.4 Trace preprocessing

Each benchmark row contributes a prompt, an `id_set`, an embedding, and a stored response. Rows are grouped by `id_set`. For a class C, the overlay sets $\widehat{\kappa}_C = |C|$ and protects $x_C = \lceil \widehat{\kappa}_C/2 \rceil$ premise units transparently. The residual count is

$$n_C = \widehat{\kappa}_C - x_C. \tag{S1.1}$$

For every query embedding z_i in class $C(i)$, the positive similarity is its leave-one-out nearest similarity to another member of the same class,

$$s_i^+ = \max_{j \in C(i),\ j \neq i} \langle \bar{z}_i, \bar{z}_j \rangle, \tag{S1.2}$$

Table S1.1: Trace configuration and calibrated thresholds. Displayed thresholds are rounded; policy decisions use the full-precision values retained in the result artifacts.

Quantity	SearchQueries	LmArena
Rows used	20,000	20,000
Benchmark classes	6,113	3,372
Static quantile q	0.70	0.90
τ_{static}	0.9851999	0.9999999
τ_{ad}	0.9481447	0.9813007
Mean s_i^+	0.9373263	0.9618819
Mean s_i^-	0.8100051	0.7080125

where $\bar{z}$ denotes unit normalization; singleton classes receive $s_i^+ = 1$. The foreign-class score is the maximum cosine similarity to a foreign-class centroid,

$$s_i^- = \max_{C \neq C(i)} \langle \bar{z}_i, \bar{\mu}_C \rangle, \qquad \mu_C = \frac{1}{|C|} \sum_{j \in C} z_j. \tag{S1.3}$$

The singleton value $s_i^+ = 1$ represents a designated cached response, not a measured leave-one-out match. Foreign-centroid scores are used for threshold calibration. The score definitions require nonzero query embeddings and class centroids. They are undefined for zero-norm vectors.

S1.5 Threshold calibration

The static operating point is a quantile of the positive-similarity distribution,

$$\tau_{\text{static}} = Q_q(\{s_i^+\}). \tag{S1.4}$$

The initial configuration used $q = 0.90$ for both benchmarks. For SearchQueries, this quantile reaches the singleton convention value $s_i^+ = 1$, so the reported SearchQueries operating points use $q = 0.70$. For LmArena, $q = 0.90$ gives $\tau_{\text{static}} = 0.9999999$, which is also nearly saturated: its static own-class hits are singletons or near-exact duplicates. These static quantiles are exploratory, data-dependent reporting choices made after inspection of the score distributions. They are not held-out calibrations and are not claimed to define optimal operating points. Let $\ell = \min\{\min_i s_i^+, \min_i s_i^-\}$, $a = \max\{0, \ell - 10^{-3}\}$, and $\mathcal{T} = \{a + \frac{j}{120}(1 - a) : j = 0, \ldots, 120\}$ the fixed 121-point uniform grid on $[a, 1]$. The adaptive threshold is selected by

$$\tau_{\text{ad}} = \min \left\{ \tau \in \mathcal{T} : \frac{1}{N} \sum_i \mathbf{1}[s_i^- \geq \tau] \leq \delta \right\}. \tag{S1.5}$$

The reported calibrations have nonempty feasible sets. Because the own-hit count is nonincreasing in τ, this rule maximizes the number of own hits over the feasible grid points and resolves ties toward the smallest threshold. The empirical quantile Q_q uses linear interpolation between adjacent order statistics: for ordered values $s_{(1)} \leq \cdots \leq s_{(N)}$, with $h = 1 + (N - 1)q$, $j = \lfloor h \rfloor$, and $\lambda = h - j$, $Q_q = (1 - \lambda)s_{(j)} + \lambda s_{(\min\{j+1,N\})}$. The implementation clips positive scores to $[0, 1]$ before computing this quantile.

S1.6 Stress-test loop

Algorithm S1.1 Query-local premise-erasure audit with shared policy masks

Require: class labels $C(i)$, scores s_i^+, s_i^-, thresholds $\tau_{\text{static}}, \tau_{\text{ad}}$, erasure grid $\mathcal{E}$, seed s
Ensure: policy counts, error components, and all-request availability estimates

for each class C **do**
  set $\widehat{\kappa}_C \leftarrow |C|$, $x_C \leftarrow \lceil \widehat{\kappa}_C/2 \rceil$, $n_C \leftarrow \widehat{\kappa}_C - x_C$
end for
for each $\varepsilon \in \mathcal{E}$ **do**
  initialize the random generator with seed s
  for each request i **do**
    draw independent erasure indicators for its $n_{C(i)}$ exposed leaves
    set $o_i \leftarrow 1$ if all these leaves survive, else $o_i \leftarrow 0$
  end for
  for $\tau \in \{\tau_{\text{static}}, \tau_{\text{ad}}\}$ **do**
    set $h_i^+(\tau) \leftarrow \mathbf{1}[s_i^+ \geq \tau]$ for every request
    set $h_i^-(\tau) \leftarrow \mathbf{1}[h_i^+(\tau) = 0,\ s_i^- \geq \tau]$
    baseline serves when $h_i^+(\tau) = 1$ or $h_i^-(\tau) = 1$
    count premise errors $\sum_i h_i^+(\tau)(1 - o_i)$ and false hits $\sum_i h_i^-(\tau)$
  end for
  static oracle-filtered arm serves when $h_i^+(\tau_{\text{static}}) = 1$
  proof-valid arm serves when $h_i^+(\tau_{\text{static}})\, o_i = 1$
  aggregate counts, rates, and $\widehat{R} = N^{-1} \sum_i o_i$
end for

The shared-mask counts satisfy

$$N_{\text{PV}} = N_{\text{own,static}} - N_{\text{premise,static}},$$
$$N_{\text{static}} - N_{\text{PV}} = N_{\text{false,static}} + N_{\text{premise,static}}.$$

The implementation asserts both identities before writing summaries.

The generator is reset to the same seed at each erasure rate. The grid points therefore reuse the random-number stream and should not be treated as independent simulation replicates. Policy comparisons at a fixed rate use identical survival indicators.

The policies prefer an own-class candidate whenever it clears the threshold and consider a foreign candidate only otherwise. This oracle-aided accounting rule uses benchmark class labels and is not unified-pool nearest-neighbor serving. Own-class filtering removes false hits; availability gating removes the remaining premise errors.

The foreign score uses class centroids, so the connected-component argument in Section S1.8 does not apply here. Erasure draws are query-local: requests in the same class share an exponent but not a survival indicator. The audit therefore estimates request-level availability, not joint reliability under shared source loss.

S1.7 Class-aware policy accounting

The class-aware benchmark accounting of main-text Section 3.4 distinguishes two disjoint error components among served requests:

- a *false hit*: a candidate from a different benchmark class is served, irrespective of its evidence availability;
- a *premise error*: an admissible candidate is served without an available certificate under its evidence contract.

Table S1.2: Trace operating points. Errors use served requests as the denominator. The oracle arm removes foreign serves from the static baseline; PV applies both own-class filtering and availability gating.

Dataset	ε	Static error	Static + oracle error	Adaptive error	PV served
SearchQueries	0.1	0.1451	0.1447	0.2082	5132
	0.3	0.3272	0.3268	0.4748	4039
	0.5	0.4341	0.4338	0.6249	3397
LmArena	0.1	0.2951	0.2941	0.3096	1452
	0.3	0.6500	0.6495	0.6872	721
	0.5	0.8184	0.8182	0.8708	374

Table S1.3: Benchmark policy mechanisms for designated evidence. "Excluded" refers to unavailable-evidence reuse under the declared contract and checked audit state, not to all answer errors.

Policy	Similarity	Dependency identity	Current availability check	Unavailable-evidence reuse
Static-threshold benchmark	yes	no	no	possible
Adaptive-threshold benchmark	yes	no	no	possible
Stored versions/hashes, no availability probes	yes	yes	no	possible
PVCacheServe certificate gate	yes	yes	yes	excluded

Foreign-class serves are assigned to the false-hit component only. This is an accounting convention under the benchmark contracts, not a complete taxonomy of factual or semantic errors. Certificate gating removes the second component relative to the checked audit state by declining reuse when the availability check fails. It does not determine benchmark-class admissibility. The class-aware benchmark evaluations therefore report own-class filtering separately from the certificate check.

Table S1.3 compares designated-evidence policy mechanisms rather than matched executions of existing systems.

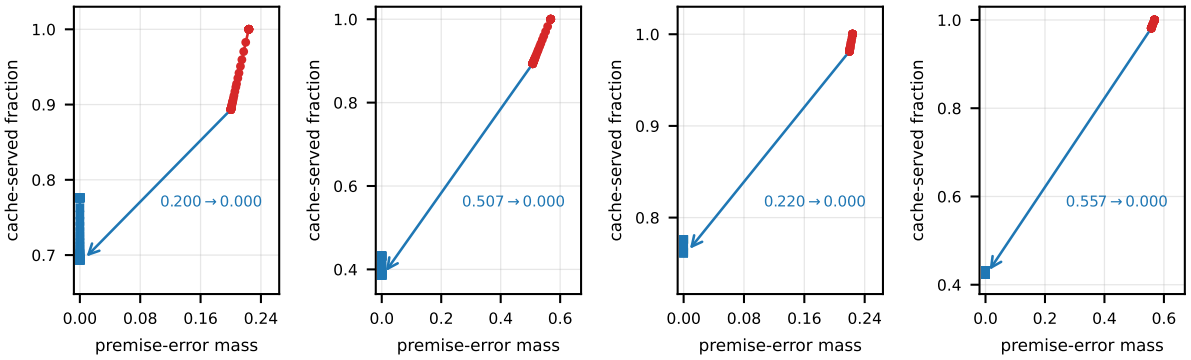
Section S1.8 illustrates the conversion of premise-error mass into declined reuse under a class-aware benchmark policy.

S1.8 Error–throughput operating curves

We sweep the serving threshold $\tau \in [0.50, 0.99]$ using HotpotQA connected components constructed as in Section S1.1, at $\tau_c = 0.65$ and 0.75. These thresholds give 6,874 and 7,331 components, with 10.7% and 1.9% of questions in multi-member components. The groups define an assumed reuse contract, not independently verified answer-equivalence classes.

For non-singleton requests, the positive score is the best leave-one-out same-component similarity. Singletons receive the convention $s_i^+ = 1$, representing an available designated cached response rather than an observed leave-one-out semantic hit (89.3% of questions are singletons at $\tau_c = 0.65$, 98.1% at $\tau_c = 0.75$). The gate checks the requesting question's designated evidence set.

Here s_i^- is the maximum pairwise similarity to a question outside the request's component, rather than the foreign-centroid score of Section S1.4. Every non-singleton vertex has an inci-



(a) τ_c=0.65, ε=0.1. (b) τ_c=0.65, ε=0.3. (c) τ_c=0.75, ε=0.1. (d) τ_c=0.75, ε=0.3.

Figure S1.2: Expected HotpotQA policy-accounting curves over $\tau \in [0.50, 0.99]$. Circles denote similarity-only reuse and squares denote gated reuse. Arrows connect the policies at $\tau = 0.9$. The grouping and own-class-priority rule exclude foreign serves in this audit; most requests are singletons represented by a designated-response convention. These are evidence-availability operating curves, not a matched semantic-cache serving frontier.

dent within-component edge with similarity at least τ_c; singletons have $s_i^+ = 1$ by convention. Hence $s_i^+ \geq \tau_c$ for every request. Conversely, $s_i^- < \tau_c$, because any pair reaching τ_c would belong to the same component.

Under the own-class-priority accounting rule of Section S1.6, an own-class candidate therefore passes whenever $\tau \leq \tau_c$. When $\tau > \tau_c$, no foreign candidate passes. Thus no foreign serve occurs anywhere in this sweep.

Let $H_i(\tau) = \mathbf{1}[s_i^+ \geq \tau]$ and $p_i(\varepsilon) = (1-\varepsilon)^{e_i}$. The expected operating points are

$$T_{\mathrm{SO}}(\tau) = \frac{1}{N}\sum_i H_i(\tau), \qquad E_{\mathrm{SO}}(\tau,\varepsilon) = \frac{1}{N}\sum_i H_i(\tau)\,[1-p_i(\varepsilon)],$$
$$T_{\mathrm{PV}}(\tau,\varepsilon) = \frac{1}{N}\sum_i H_i(\tau)\,p_i(\varepsilon), \quad E_{\mathrm{PV}}(\tau,\varepsilon) = 0.$$

Thus $E_{\mathrm{SO}} = T_{\mathrm{SO}} - T_{\mathrm{PV}}$: premise-error mass becomes declined reuse mass. Here throughput is the cache-served request fraction, and error is unconditional request-level mass. The main curves are analytical; Bernoulli checks at $\tau = 0.75$ and 0.9 verify the serving fractions. Singletons dominate both groupings, so the curves primarily measure designated-response availability. They are not a matched semantic-cache performance frontier and do not test transfer of a different cached response's certificate to the request.

At $\tau = 0.9$, baseline error ranges from 0.200 to 0.220 at $\varepsilon = 0.1$ (Figure S1.2(a,c)) and from 0.507 to 0.557 at $\varepsilon = 0.3$ (Figure S1.2(b,d)). For example, at $\tau_c = 0.65, \varepsilon = 0.1$, gating reduces throughput from 0.893 to 0.693 while removing the 0.200 premise-error mass (Figure S1.2(a)).

S1.9 Evidence-count perturbations

We perturb HotpotQA evidence counts using $D_q \sim \mathrm{Binomial}(e_q, p_{\mathrm{miss}})$ and $A_q \sim \mathrm{Poisson}(p_{\mathrm{add}} e_q)$, setting

$$\widehat{e}_q = \max\{1, e_q - D_q + A_q\}.$$

At $p_{\mathrm{miss}} = p_{\mathrm{add}} = 0.3$, the mean absolute count error is 0.77, while the absolute change in workload-average predicted availability is at most 0.038 over the reported erasure grid. Omissions alone increase this average by up to 0.127, whereas additions alone decrease it by up to

0.082. Balanced perturbations can therefore mask opposing count errors in an aggregate availability comparison. The omission and insertion draws are mutually independent and independent across questions; reported values are a single seeded realization (seed 42) per $(p_{\text{miss}}, p_{\text{add}})$ pair, not averages over repetitions.

S1.10 Small-schema state-enumeration check

We check main-text Lemma 4.1 on 400 generated conjunctive DAGs (seed 20261022; `experiments/16_dag_correctness_test/`). Leaf and internal counts are drawn independently and uniformly from $\{1, \ldots, 5\}$ and $\{1, \ldots, 6\}$. Vertices are ordered with leaves first. Each internal vertex selects one to three earlier parents uniformly without replacement, subject to the number available. When there are at least two internal vertices, the sink also takes its immediate predecessor as a parent. Leaves need not reach the sink.

For schema g, let L_g be its leaves and C_g all non-sink vertices. For every

$$(\widetilde{B}, S) \in 2^{L_g} \times 2^{C_g},$$

separate closure and reverse-reachability routines compare

$$q \in \mathrm{Cn}_g(\widetilde{B} \cup S) \quad \text{and} \quad D_{\exp}(q, S) \subseteq \widetilde{B}.$$

They agree in all 990,560 states. Enumeration is exhaustive within each sampled schema, not over all DAGs of these sizes. This checks deterministic closure semantics, not support sufficiency or serving performance.

S2 Numerical Coding, Shared-Root, and Alternative-Family References

S2.1 Conditional protection references

Finite-size coded storage. These calculations instantiate the companion's coded payload-recovery comparison [7] on annotated footprints. We apply the designated-payload model to annotated evidence counts, treating each evidence item as one equal-cost packet. These are storage calculations, not encoder–decoder executions. At $\varepsilon = 0.1$ and $\delta = 0.02$, mean normalized pure-parity storage is 0.538 for annotated HotpotQA evidence counts and approximately 0.301 when the footprint is the number of records in each question's supplied dataset context. The latter uses dataset contexts, not the retrieved record sets of Section S1.2. Fixed sizes $\kappa = 20, 50, 100$ give $0.25, 0.20, 0.17$, compared with the asymptotic value 0.1 (Figure S2.1(a)). For the annotated HotpotQA sizes $\kappa_q \in \{2, \ldots, 8\}$, the required parity count is one to three packets (Figure S2.1(b)). Table S2.1 reports availability under a size-dependent parity budget.

Shared-root storage. Questions are connected when their gold evidence shares a document in HotpotQA or an identical paragraph in MuSiQue. The audit includes 742 non-singleton HotpotQA components containing 1,675 questions and 293 non-singleton MuSiQue components containing 2,182 questions. Within each component, let U be the set of question–document/paragraph occurrences, and let $\mathcal{R}$ contain the candidate shared roots that cover at least two occurrences. An individual occurrence copy costs one packet unit, and each root costs $\rho \geq 0$ units. The audit assumes that a selected root may replace all occurrences with its identifier.

If root d covers $k_d \geq 2$ occurrences, selecting it costs ρ instead of k_d individual copies. Because candidate-root coverage sets are disjoint, these choices are separable:

$$\mathrm{OPT}(\rho) = |U| - \sum_{d \in \mathcal{R}} (k_d - \rho)^+.$$

Table S2.1: Question-weighted mean designated-payload availability under a size-dependent parity budget. Source-only values average $(1-\varepsilon)^{\kappa_q}$; MDS-protected values average $\Pr[\text{Binomial}(\kappa_q, \varepsilon) \le B_q]$, where $B_q = \max\{1, \text{round}(0.3\kappa_q)\}$. These means do not impose a per-question reliability target. Packets have equal normalized cost, and code existence is assumed.

Dataset	ε	Source-only	MDS-protected	Mean κ_q
HotpotQA	0.1	0.776	0.983	2.43
	0.3	0.432	0.860	
	0.5	0.203	0.654	
MuSiQue	0.1	0.759	0.977	2.65
	0.3	0.402	0.827	
	0.5	0.179	0.598	

Occurrences without a selected covering root retain individual copies. At $\rho = 0.5$, stipulated-cost savings relative to replicated occurrence storage are 44.6% for HotpotQA and 71.9% for MuSiQue (Figure S2.2). Greedy root selection matches this closed form. Each selected root saves $k_d - \rho = (k_d - 1) + (1 - \rho)$ units, combining a sharing gain with the stipulated per-object cost difference; the reported savings therefore depend on both genuine sharing and the choice of ρ. These are stipulated costs and accepted substitutions under the $k_d \ge 2$ candidate restriction, not measured byte sizes, validated semantic support, or an advantage over globally deduplicated premise storage.

Section S2.2 reports larger-footprint equal-budget curves.

Hybrid storage accounting. For each designated certificate of size κ_q, the nominal transparent fraction η gives $x_q = \text{round}(\eta\kappa_q)$ and $m_q = \kappa_q - x_q$. The required residual parity count is $r_q = r^*_{\text{MDS}}(m_q, \varepsilon, \delta)$. For N designated certificates, the plotted quantity is

$$\overline{h}(\eta) = \frac{1}{N} \sum_q \frac{x_q + r_q}{\kappa_q}.$$

This is a question-weighted mean of normalized storage, not a ratio of total stored bytes to total payload bytes.

S2.2 Equal-packet-budget coding references

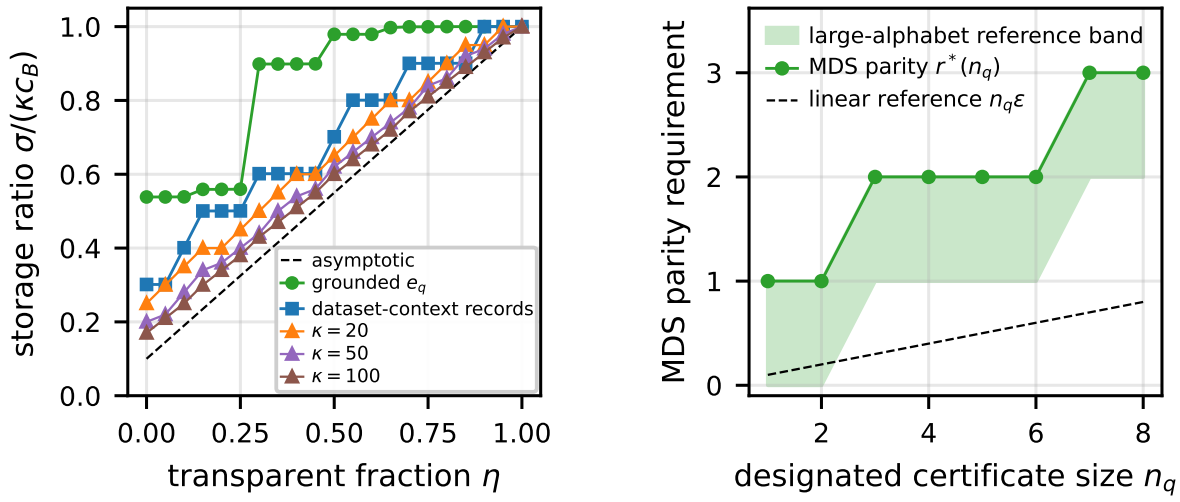
For an integer packet budget $B \in \{0, \ldots, \kappa\}$, reliable transparent storage of B designated packets gives availability

$$R_{\text{copy}}(\kappa, B, \varepsilon) = (1-\varepsilon)^{\kappa - B}.$$

Storing B reliable MDS parity packets instead gives

$$R_{\text{MDS}}(\kappa, B, \varepsilon) = \Pr[\text{Binomial}(\kappa, \varepsilon) \le B].$$

Both expressions assume a designated payload set, fixed eligibility, independent source erasures, and reliable equal-cost protected packets. The MDS expression additionally assumes code existence. Figure S2.3(a) compares MDS-parity and transparent-copy availability for the same designated payload set, and Figure S2.3(b) shows their difference. At $\varepsilon = 0.1$ and $B = \text{round}(0.3\kappa)$, availability gains are approximately $0.16, 0.51, 0.77, 0.95$ for $\kappa = 3, 10, 20, 40$. Pure parity is optimal within the ideal MDS hybrid family (main-text Remark A.1). Table S2.1 applies a size-dependent parity budget to the annotated footprints. The larger sizes in Figure S2.3 are analytical references, not measured certificates.



(a) Coded regime map.

(b) MDS parity requirement.

Figure S2.1: Designated-payload storage references at $\varepsilon = 0.1$ and $\delta = 0.02$. (a) Question-weighted mean normalized MDS-hybrid storage. (b) Required parity counts for payload size $n_q = \kappa_q$, the mean-erasure reference $n_q\varepsilon$, and the large-alphabet reference band $[(r^*_{\mathrm{MDS}}(n_q, \varepsilon, \delta) - 1)^+, r^*_{\mathrm{MDS}}(n_q, \varepsilon, \delta)]$. The band's lower edge is a companion-theory reference, not a converse proved here. For packets of c_B bits, that theory's finite-alphabet converse instead uses $\delta + s_{c_B}$, where $s_{c_B} = (2^{c_B} - 1)^{-1}$ and $\delta + s_{c_B} < 1$ [7]. No encoder–decoder implementation is evaluated.

S2.3 Heuristic alternative evidence families

We add singleton alternatives whose sentences contain the answer string and lie outside the gold set. The strict construction searches gold supporting documents; the loose construction also searches distractors. Neither construction validates semantic sufficiency. The strict construction adds alternatives for 1,099 of 7,405 questions (14.8%); the loose construction does so for 2,476 (33.4%). Mean family sizes over all questions are 1.21 and 2.24, respectively. Each measured curve is checked against the family bounds of main-text Proposition A.2 for its own family at every reported erasure rate (Figure S2.4). The strict construction gains 0.069 over survival of the designated gold certificate at $\varepsilon = 0.4$; this gold-only comparison is distinct from the minimum-residual-certificate lower bound of main-text Proposition A.2.

For the constructed families, availability also has a closed form. Let a_q be the number of distinct admitted singleton leaves outside the gold set. For $0 < \varepsilon < 1$, independent leaf erasure gives

$$R_q^{\mathrm{family}}(\varepsilon) = 1 - \left[1 - (1 - \varepsilon)^{e_q}\right] \varepsilon^{a_q}.$$

The formula extends to $\varepsilon = 0$ by setting availability to one. Simulation averages differ from the exact probabilities by less than 0.0012 on the grid.

S3 Descriptive Snapshot Dynamics

This section provides descriptive evidence-maintenance context. Snapshot changes are not calibrated erasure probabilities and do not establish cached-answer errors.

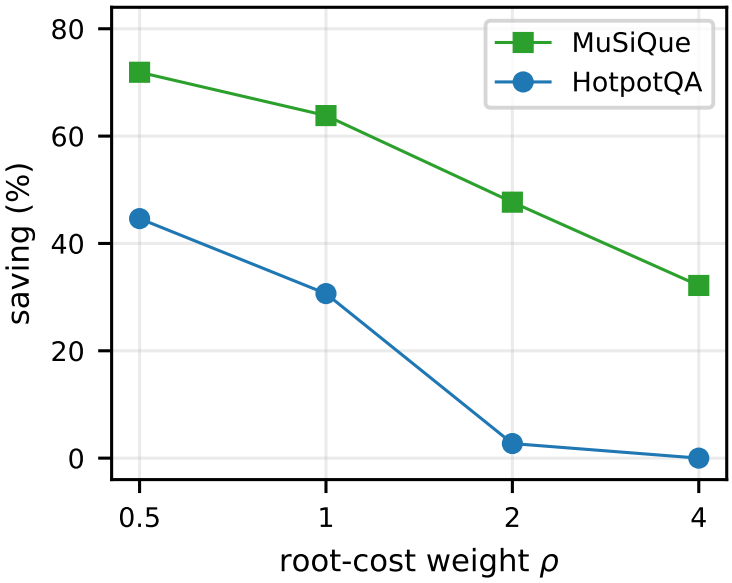


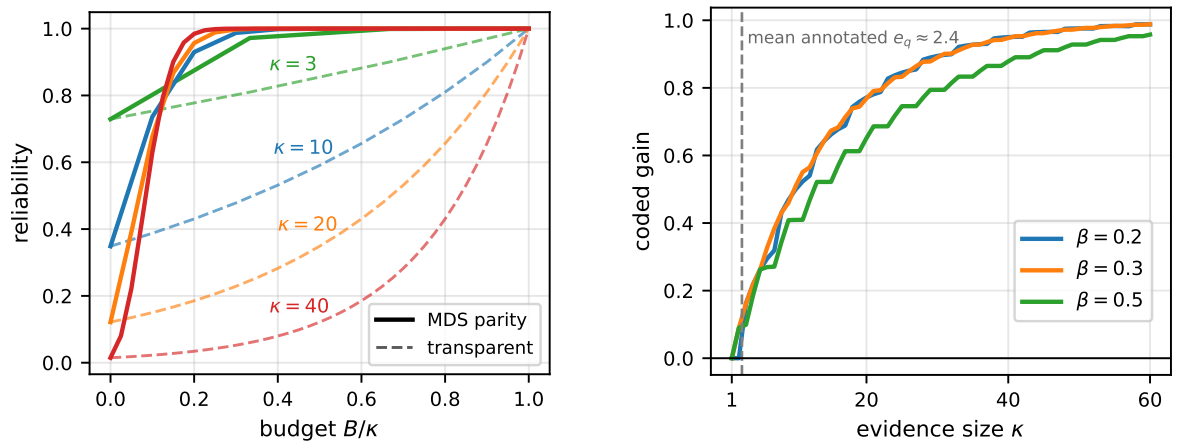
Figure S2.2: Full-coverage shared-root savings relative to replicated occurrences in 742 HotpotQA and 293 MuSiQue components. Root cost ρ is in packet units. The reference is not globally deduplicated payload storage.

S3.1 Snapshot changes

A longitudinal snapshot audit. We measure text and structured-claim changes relevant to evidence maintenance. We use five monthly English Wikipedia snapshots from 2026-04-01 to 2026-08-01 and four Wikidata snapshots over the same interval [8], and track a frozen panel of 2,000 entities sampled (seed 42) from the subjects of the TemporalWiki lifelong benchmark [9], a population selected for temporal volatility. Page text is recovered for 1,883 entities in every snapshot, of which 1,749 yield usable extracted sentences. Structured claims are recovered for 1,739 entities under the selected truthy view. As a complementary change reference we use the legacy split of WikiFactDiff [10], comprising 327,688 update instances between Wikidata snapshots of 2021-01-04 and 2023-02-27. The annotation-based rule in main-text Appendix B.1 classifies 32,873 instances (10.0%) as value replacements. This dataset and the 2026 panel differ in population, time horizon, and change definition: they provide complementary measurements, not replicate estimates of one common erasure rate. The snapshot measurements are descriptive evidence-maintenance indicators and are not used to calibrate ε: their observation units, populations, and dependence structures differ from those of the controlled erasure experiments.

Observed text and claim changes. Table S3.1 and Figure S3.1(a) report changes relative to the 2026-04-01 base snapshot. The fraction of normalized base sentences absent from the later text rises from 1.61% after one month to 5.21% after four months; such disappearance includes rewriting and does not necessarily imply loss of the underlying fact. At four months, 58.7% of pages have different wikitext at the two snapshot endpoints. The lead-5 indicator flags loss among a page's first five extracted sentences, or all sentences on shorter pages, and fires on 13.7% of pages. These endpoint comparisons do not identify all edits made during the interval.

At the structured layer, 8.0% of entities have at least one projected base truthy claim, other than `P421`, absent from the selected representation after four months. This property accounts for approximately 26% of raw removal records and is excluded to separate the identified



(a) Reliability. (b) Coded gain.

Figure S2.3: Equal-budget protection at $\varepsilon = 0.1$. (a) MDS-parity and transparent-copy availability. (b) Their difference at $B = \text{round}(\beta\kappa)$ for budget fractions β (the text reports $\beta = 0.3$); the dashed line marks mean annotated evidence size. These are equal-cost packet references.

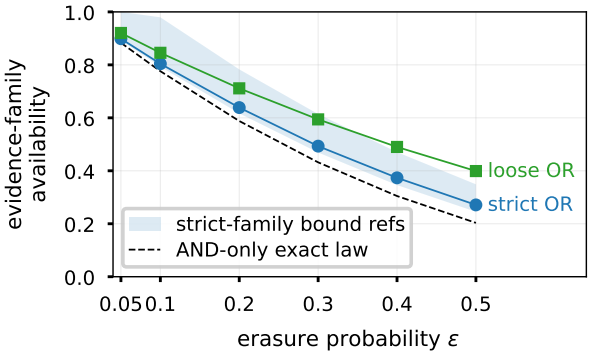


Figure S2.4: Availability of gold-only, strict, and loose heuristic certificate families on HotpotQA (20 repetitions, seed 42). Shading gives the strict-family minimum-size lower bound and unclipped average upper reference $N^{-1}\sum_q K_q(1-\varepsilon)^{e_{\min,q}}$, where K_q is family size.

timezone-related change from the reported non-`P421` measure. The percentage is entity-level incidence, not a per-statement erasure probability. Disappearance from the selected representation may reflect deletion, replacement, or changes in rank-based selection and need not imply factual falsity.

Figure S3.1(b) shows non-monotone WikiFactDiff replacement fractions across popularity groups, peaking at approximately 21.1% in decile 3 (zero-based indexing).

Text results use the `text` records in `eps_t.json`; claim results use `eps_stmt_v2.json`. The earlier `stmt` records in `eps_t.json` are not used in Table S3.1.

S4 Extended Replay Tables and Case Analyses

S4.1 Reuse decisions and answer-level trade-offs

From snapshot changes to reuse decisions. The descriptive changes in Section S3 do not establish answer errors. The replay checks a cache built at 2026-04-01 against the 2026-08-01 state, using a shared similarity screen and different dependency contracts. Non-serves enter

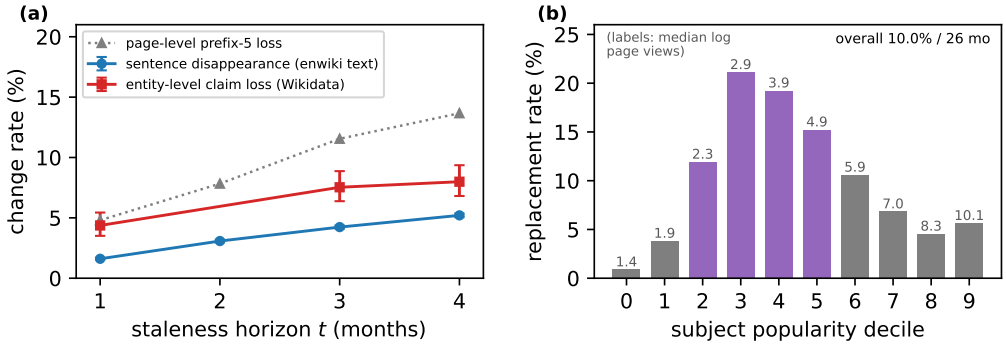


Figure S3.1: Descriptive change measurements. (a) Base-sentence disappearance, entity-level truthy-view claim loss excluding `P421`, and page-level lead-5 sentence loss. Sentence shading gives nominal Wilson 95% intervals without within-page clustering adjustment. (b) WikiFact-Diff annotation-defined replacement fractions by popularity decile (0–9); labels give median log page views.

Table S3.1: Changes from the 2026-04-01 baseline. Text measures use 1,749 pages and 86,441 base sentences; truthy-view loss uses 1,739 entities and excludes `P421`. Lead-5 loss means any disappearance among the first $\min\{5, L_p\}$ extracted sentences of page p, where L_p is its base-snapshot sentence count. Brackets give nominal Wilson 95% intervals; sentence intervals ignore within-page clustering.

t (mo)	wikitext differs	sentence loss	lead-5 loss	truthy-view loss
1	25.8%	1.61% [1.53,1.70]	4.80% [3.90,5.91]	4.37% [3.51,5.44]
2	39.7%	3.08% [2.97,3.20]	7.83% [6.66,9.19]	–
3	51.7%	4.24% [4.10,4.37]	11.55% [10.14,13.13]	7.53% [6.38,8.87]
4	58.7%	**5.21%** [5.06,5.36]	**13.66%** [12.13,15.35]	**7.99%** [6.81,9.36]

current-evidence fallback or abstention. Panel construction, screening, and annotation follow main-text Section 6.

Under the annotation protocol of main-text Section 6.4, 141 base answers are correct, one wrong, seven incomplete, and five indeterminate. Of the 154 paraphrases, 151 are faithful; the three relation-drift cases appear in Table S4.2.

Wikipedia-sentence support is full for 109 entries and partial for 45. The supplied sentences do not cover every designated claim, even though the generation evidence also includes structured claims. This coverage assessment is separate from the sentence-identity condition: later disappearance of a required sentence can trigger a decline without establishing that the answer has become wrong.

These three hits are wrong for the incoming requests and are excluded from the confirmed-correct denominator, leaving 118 hits. Their rejection reflects unavailable dependencies, not detected intent drift.

At the target snapshot, 126 cached answers remain correct for their cached questions (3 of them answer meaning-drifted incoming queries; Table S4.2), 3 are wrong, 18 incomplete, and 7 indeterminate. Attribution separates temporal decay from pre-existing defects: all three wrong answers are invalidated by observed changes, and six of the eighteen incomplete answers are partially invalidated by changes, while the remaining twelve reflect base-period generation gaps or pure value additions that overturn no asserted fact. We call these $3 + 6 = 9$ entries *decayed*; all nine lie in the dependency-stale arm, and eight of them are similarity hits. Conversely, 33 of the 50 dependency-stale entries (66%) remain fully correct at the answer level: the removed claims were values the answer never asserted, synonym-entity rebuilds, or precision

Table S4.1: Controlled snapshot cache-reuse replay (154 entries: 50 dependency-stale, 104 claim-intact; similarity screening accepts 147: 43 stale, 104 intact). "Not cache-served" sums similarity misses and gate declines; fallback regeneration of these requests is reported in Section S4.2. Claim staleness denotes absence of a designated base claim value from the target projected eligible-claim view; Table S4.3 reports the independently annotated answer-level outcomes. The asserted-claims variant re-verifies only the designated claims named by the cached answer; its 7 stale serves are all annotated correct at the answer level.

reuse policy	cache-served	stale served	intact hits declined	not cache-served
similarity-only	147	43	0/104	7
strict dependency gate	89	0	15/104	65
claims + majority sentences	96	0	8/104	58
claims only	104	0	0/104	50
asserted claims only	111	7	0/104	43

Table S4.2: The three paraphrases annotated as unfaithful (A3). All ask about actors or voice actors, whereas the cached answers name composers. Similarities are 0.866–0.892, above τ. All three entries also lose their sole designated asserted claim, so the gated contracts decline them for evidence reasons, not detected intent drift.

paraphrase (unfaithful, A3)	cached answer	target status for cached question
"Who acted in *The Fifth Element*?"	Éric Serra	correct (composer)
"Which actor or actress starred in the film *As Good as It Gets*?"	Hans Zimmer	correct (composer)
"Which actors provided the voices for *Cars 3*?"	Randy Newman	correct (claim moved to `P86`)

placeholders. Dependency staleness therefore flags more than five times as many entries as show answer-level decay on this panel (50 versus 9), qualitatively consistent with FreshCache's finding that most hash-detected web changes do not affect answer correctness [11].

Results and interpretation. Table S4.1 reports dependency-level routing outcomes. The similarity-only policy serves 43 dependency-stale entries among its 147 hits (29.3% of what it serves; Wilson 95% interval $[22.5, 37.1]$). The strict, majority, and claims-only gates decline all 43 dependency-stale hits by construction: the same projected claim membership defines staleness and gate acceptance. The asserted gate serves seven of them because the missing values are outside its lexical asserted set; all seven answers are labeled correct. The replay verifies execution of these contracts and measures their additional declines, not independent staleness detection.

The strict gate additionally declines 15 of 104 claim-intact hits (14.4% $[8.9, 22.4]$) because at least one designated sentence string is absent from the target page; in all 15 cases the designated claims survive, so the sentence-identity requirement alone triggers these declines. String absence by itself establishes neither that the underlying fact changed—the four-month sentence-disappearance rate of 5.2% in Table S3.1 includes rewriting—nor that every declined answer remains correct. Relaxing the sentence conjunct to a majority reduces these declines to 8 (7.7%), and the claims-only contract declines none. On this panel, whose labels are defined by claim disappearance, sentence-identity requirements therefore add declines without adding interceptions; this quantifies the price of a stricter contract, not the semantic redundancy of the sentences.

Answer correctness is assessed separately from the gate verdict. Annotators see two-period claims, extracted sentences, and survival marks, but not gate outputs or explicit staleness labels. Dependency staleness is therefore inferable from the evidence sheet. The answer labels

Table S4.3: Answer-level routing outcomes on 147 similarity hits. The reporting groups are eight temporally decayed hits (three wrong and five partially invalidated), three meaning-drifted hits, 118 confirmed-correct hits, twelve change-independent incomplete hits, and six indeterminate hits. Decay and drift are attribution tags, not general quality categories. Correct-hit declines use 118 as denominator; declines of incomplete and indeterminate hits are reported separately. All four contracts decline the eleven decayed-or-drifted hits on this panel.

reuse policy	served	decayed served	drifted served	correct declined	incompl. declined	indet. declined
similarity-only	147	8 (3+5)	3	0/118	0/12	0
strict dependency gate	89	0	0	38/118	4/12	5
claims + majority sentences	96	0	0	33/118	3/12	4
claims only	104	0	0	25/118	3/12	4
asserted claims only	111	0	0	18/118	3/12	4

are not defined by claim disappearance, although the assessment is not blind to the underlying evidence state. Table S4.3 reports the answer-level outcomes.

Similarity-only reuse serves eight decayed answers among 147 hits (5.4%; Wilson 95% interval $[2.8, 10.4]$): three are wrong and five partially invalidated. It also serves the three meaning-drifted hits, giving eleven decayed-or-drifted serves (7.5%; $[4.2, 12.9]$). Six are wrong for the incoming request and five are partially invalidated.

All four contracts decline these eleven hits. Among the 118 confirmed-correct hits, claims-only, majority-sentence, and strict reuse decline 25, 33, and 38, respectively: 21.2% $[14.8, 29.4]$, 28.0% $[20.7, 36.7]$, and 32.2% $[24.4, 41.1]$. These brackets are nominal Wilson 95% intervals.

Entity-cluster bootstrap intervals use 10,000 resamples of the 136 entities represented among the 147 similarity hits (seed 42). The intervals are $[2.8, 12.8]$% for the similarity-only decayed-or-drifted served share and $[23.5, 40.8]$% for strict-gate confirmed-correct declines. These are marginal intervals for individual rates, not paired intervals for policy differences.

On this panel, sentence requirements add correct-hit declines without additional observed interceptions. The lexical asserted set averages 1.69 values, compared with 2.79 designated claims per entry. The asserted-claims contract further reduces correct-hit declines from 25 to 18 without losing an observed decay interception. These outcomes compare different evidence contracts; they do not establish that weaker contracts are generally preferable.

S4.2 One-shot regeneration and delivery outcomes

One-shot fallback. The strict contract routes 65 requests to fallback: seven similarity misses and 58 gate declines. Fallback first checks for usable target-snapshot evidence. When available, the generation input contains the property's current values admitted by the claim projection and up to three target-page sentences mentioning them. Generation uses the base-cache model configuration and value-mention acceptance rule.

Of the 65 requests, 51 receive accepted regenerated answers. The remaining 14 have no value admitted by the claim projection for the property in the target entity view and abstain without generation. They include all three meaning-drifted queries, so none receives a response under this one-shot strict policy. All nine temporally decayed entries receive accepted regenerations whose answers differ from the cached responses and mention current values. The strict policy therefore delivers 89 cached and 51 regenerated responses, with 14 abstentions.

Two annotators independently assess the 51 regenerated answers for target-period correctness (R1) and Wikipedia-sentence coverage (R2), with disagreements adjudicated against the target-period values and evidence sentences. Agreement is 92.2% on each question. After adjudication, 43 answers are correct and eight incomplete, with none wrong or indeterminate; the second annotator's raw correctness counts are 45 and six. Wikipedia-sentence coverage is full

Table S4.4: Unified strategy view over all 154 requests, in mutually exclusive delivery-quality categories (each row sums to 154). *Wrong* = answer annotated wrong at the target snapshot (3) or answering a meaning-drifted incoming query (3); *incomplete* includes the 5 partially invalidated answers. Temporal decay (8 hits) and query drift (3 hits) are attribution tags on a separate axis: every gated contract avoids all 11 tagged serves, similarity-only serves all 11 (plus regenerating the one decayed miss). Regenerated answers carry their adjudicated answer-level labels (none wrong, none indeterminate); all policies share the same archived fallback realization per request.

policy	correct	incomplete	wrong	indet.	abstain
similarity-only	119	17	6	6	6
strict	123	16	0	1	14
claims + majority sentences	121	17	0	2	14
claims only	122	16	0	2	14
asserted claims only	122	16	0	2	14

for 19, partial for 11, and absent for 21. This coverage measure is distinct from correctness: value-mention acceptance establishes lexical overlap with at least one supplied value, and generation also uses structured claims, so lexical acceptance does not establish support for every answer assertion.

The 89 cached deliveries comprise 80 correct, eight incomplete but not decayed, and one indeterminate answer. Together with regeneration, final delivery comprises 123 correct, 16 incomplete, one indeterminate, and 14 abstentions. No temporally decayed cached answer is served, and no regenerated answer is annotated wrong. Six of the seven similarity misses abstain because the target claim view has no value for the property, even though relevant answer text remains in Wikipedia. The declared claim-sourced fallback does not use that text alone.

Unified strategy view. Table S4.4 uses mutually exclusive delivery-quality categories over all 154 requests, keeping wrong and incomplete separate (temporal decay and query drift are reported as attribution tags, not mixed into the quality counts). The policies' non-serve sets are nested: asserted $\subseteq$ claims $\subseteq$ majority $\subseteq$ strict. Consequently, the strict policy's fallback archive covers every policy's one-shot fallback demand. Each policy is evaluated using the same archived fallback output for a given request; no additional generation is required for this comparison.

Regeneration uses the cached question, which is intent-equivalent to the incoming query for the 151 faithful paraphrases. The three drifted queries abstain, so the replay does not test fallback generation for changed query intent.

Every gated policy delivers zero annotated-wrong answers and 121–123 correct answers under the primary generator, compared with six wrong and 119 correct under similarity-only reuse, while abstentions increase from six to fourteen. For the strict policy, the observed correct-delivery gain over similarity-only reuse is $4/154 = 2.6$ percentage points. Its entity-cluster bootstrap 95% interval, $[-3.9, +9.3]$ points, includes no gain and effects in either direction. The paired transitions between similarity-only reuse and the strict one-shot policy decompose this net gain: the 11 requests that gain a correct delivery are 6 incomplete, 3 indeterminate, and 2 wrong cached answers regenerated to correct, while the 7 that lose one are 4 cached-correct answers degraded to incomplete by regeneration and 3 cached-correct requests left abstaining. All 14 abstentions correspond to absence of usable current evidence under the declared fallback rule, including the three meaning-drifted queries.

Table S4.5: Rejected-set accounting for the one-shot policies on the 147 similarity hits. $\bar{c}$ and $\bar{f}$ are the cached and fallback error rates over declined hits (abstentions count as non-errors). Totals equal 23− cached errors + fallback errors and match Table S4.4.

policy	declined	cached err.	$\bar{c}$	fallback err.	$\bar{f}$	$\bar{c} - \bar{f}$	abstain	total err.
similarity-only	0	–	–	–	–	–	–	23
strict	58	15	0.26	8	0.14	0.12	8	16
claims + majority	51	14	0.27	8	0.16	0.12	8	17
claims only	43	14	0.33	7	0.16	0.16	8	16
asserted claims	36	14	0.39	7	0.19	0.19	8	16

Rejected-set accounting. Main-text Remark 5.1 compares cached and fallback errors on the additionally rejected hits. Similarity misses use the same fallback outputs under every policy and cancel. Table S4.5 counts wrong and incomplete deliveries as errors and reproduces the totals in Table S4.4. The four reductions are $7, 6, 7, 7$ errors. Every contract has a positive rejected-set margin $\bar{c} - \bar{f}$, and all share the same eight declined-hit abstentions. The identity is a one-shot comparison; write-back makes later cached answers policy-dependent.

Loss definition. The table assigns unit loss to wrong or incomplete deliveries and zero loss to abstention. It measures erroneous delivery, not service utility. The same rejected-set identity applies to other consistently defined losses; the reported counts do not identify a universally preferred policy.

S4.3 Multi-tick checks and matched endpoint assessment

The headline replay uses archived model outputs or explicit abstentions for every reachable fallback key (main-text Appendix C.1). Earlier template-fallback runs are separate mechanism checks.

Fixed-cache checks reproduce 89, 96, 104, and 111 serves. Real-driven write-back records are constructed from the actual accepted answers and issuance snapshots. Endpoint assessment joins labels to the delivered text and origin tick, including earlier write-backs. Template labels are not transferred to different outputs.

Across three update ticks, blind refresh uses 421 accepted regenerations and the asserted loop 29. Second-generator re-execution preserves these totals but changes endpoint quality (Section S4.4). The primary-generator refresh comparison remains statistically unresolved.

S4.4 Second-generator re-execution

The primary archived regenerations use `kimi-k2-turbo-preview`. We assess generator sensitivity by reissuing the archived regeneration prompts with the requested model identifier `gpt-5-mini`, accessed through the Poe API (OpenAI-compatible) on 2026-10-19—the same access path as the daily-slice GPT cache (main-text Section 6.2). The base cache, incoming requests, screening threshold, and evidence states remain fixed.

The archive replication covers the 411 keys of the merged multi-tick store, each keyed by request and origin tick (main-text Appendix C.1). Of these, 371 require a model call: 370 hold an accepted primary answer and one exhausts its acceptance retry; the remaining 40 lack usable current evidence and remain abstentions without generation. Under each generator, 174 merged-store keys supply an endpoint delivery for at least one core erasure-free policy. The sets share 173 keys and have a union of 175.

The separate one-shot archive contains 65 keys, of which 51 hold an accepted answer; the same acceptance rule and single retry apply. Calls use the archived system prompt, question, and origin-tick evidence.

Asserted-claim sets are re-extracted from the new accepted answers, certificates are rebuilt, and all five erasure-free maintenance trajectories are re-executed with per-tick gating, write-back, and reachable-key coverage checks. Reissuing gap-fill prompts does not constitute a second-generator execution of the read-fault sweep. The trajectory-based endpoint map defines the reported deliveries; direct substitution into the primary map is a diagnostic comparison only.

The primary requests disable reasoning output. The replacement requests do not explicitly set a reasoning-effort parameter, so the effective backend configuration is uncontrolled. The comparison changes the requested model and potentially its reasoning configuration together.

The retained implementation comprises `s41_gpt_regenerate.py`, `s42_gpt_compare.py`, `s46_gpt_maintenance.py`, and `s47_gpt_reexec_endpoint.py`. Their archive, acceptance, trajectory, and endpoint outputs are indexed in Section S10.

Acceptance and maintenance. Acceptance decisions agree on 409/411 merged-store records: 369 are accepted by both generators and 40 lack evidence. Each generator accepts one record rejected by the other. Both accept the same 51 one-shot outputs and abstain on the same 14 no-evidence cases.

Accepted-regeneration totals remain unchanged for all five policies (`gpt_maintenance_counts.json`). Blind refresh shifts from $[142, 139, 140]$ to $[142, 140, 139]$ regenerations per tick; page-change maintenance shifts from $[56, 70, 48]$ to $[56, 71, 47]$. At the endpoint, blind-refresh abstentions increase from 14 to 15, whereas page-change abstentions remain 13.

Human-labeled endpoint quality. Every delivered answer in the assessed second-generator multi-tick endpoint and one-shot comparisons has an answer-quality label. The assessed set contains 225 regenerated answers: 174 merged-store core endpoint answers and 51 one-shot answers. Labels are inherited for 126 answers with matching normalized text, request, and assessment context. The remaining 99 are independently labeled by the same two annotators under the same operational definitions (98.0% agreement, $\kappa = 0.961$); the first annotator adjudicates two disagreements (`annotation/gpt_agreement.json`). Base answers retain their original labels.

Table S4.6 reports the multi-tick endpoint. For one-shot strict reuse, the replacement regenerations comprise 39 correct, ten incomplete, one wrong, and one indeterminate answer. Including cached deliveries gives 119 correct, 18 incomplete, one wrong, two indeterminate, and 14 abstentions. Thus the gated loops' zero-wrong multi-tick endpoint does not extend to one-shot strict reuse.

Correct deliveries decrease in all four maintenance arms, while the static cache remains at 119. Among 98 newly labeled keys accepted by both generators, replacement answers are incomplete in 41 cases, versus 19 primary answers (`annotation/pairing_audit.json`). This key-level comparison is distinct from policy endpoint counts. The refresh-minus-asserted correct-delivery difference is -15 answers ($[-24.2, -6.2]$), versus -5 ($[-13.2, +3.0]$) under the primary archive. The within-asserted-loop primary-minus-replacement difference is 4 answers ($[0, 9]$). These comparisons condition on the fixed base cache and archived outputs, not repeated runs of either generator.

S4.5 Exploratory gate mappings

We evaluate two deterministic gate mappings, one predicate ablation, and a crossed alias control on the serving panel using archived primary-generator outputs. The overlap mapping is motivated by GroundedCache, and the equality mapping by FinCacheServe [12, 13]. Both use our claim and page representations rather than the original systems' complete gates and execution

Table S4.6: Human-labeled endpoint delivery quality over 154 requests along the re-executed second-generator (`gpt-5-mini`) trajectories. Regenerated deliveries draw on the labeled set described in the text (126 labels inherited from adjudicated identical answers and 99 assigned independently; raw agreement 98.0%, $\kappa = 0.961$; two disagreements adjudicated). Categories are mutually exclusive and each row sums to 154.

Policy	Correct	Incompl.	Wrong	Indet.	Abstain
Static cache	119	17	6	6	6
Blind refresh	105	32	1	1	15
Page-change loop	109	28	2	2	13
Strict loop	115	24	0	1	14
Asserted loop	120	18	0	2	14

stacks. They are exploratory mechanism mappings, not system reproductions. The implementation is `s48_mechanism_arms.py`, with results in `mechanism_arms.json`; the crossed control below is `s49_gate_alias_cross.py`, with results in `gate_alias_cross.json`, together with `s52_claim_equality_loop_alias.py` for the alias-enabled claim-level equality configuration.

Page-version and overlap proxy. A similarity hit is reused only if at least half of its designated claims remain in the current projected eligible-claim view, the source-page hash is unchanged since issuance, and every lexically extracted asserted value either remains in the projected claim set or occurs in the current page text. The overlap threshold of 0.5 is an experimental choice. Claim survival is not the chunk-hash Jaccard measure used by GroundedCache, and the asserted-value test is not its original answer-token support score.

Page-version and equality proxy. A similarity hit is reused only if the source-page hash is unchanged and the current projected claim-value set of the queried property exactly equals the set recorded at issuance. This is an equality rule over our evidence representation, not an implementation of FinCacheServe's full request, evidence, version, and tool/model guards.

Claim-level equality and alias control. The claim-level ablation removes the page-version condition and requires equality of the queried property's projected value set. All configurations share the requests, screening threshold, snapshots, generation archive, acceptance rule, and write-back procedure. Alias expansion is crossed with gate choice.

Table S4.8 reports accepted regenerations. Claim equality requires twelve more than asserted claims under either alias setting. Aliases reduce both totals by two and leave the page-version mappings unchanged. These are trajectory comparisons, not additive predicate effects.

Table S4.7 reports default configurations and endpoint quality. All endpoint deliveries are labeled. The exploratory equality configurations additionally use one intermediate write-back absent from core endpoint delivery; both annotators label it incomplete. Endpoint labels are not transferred between configurations merely because regeneration totals agree.

S5 Simple Serve-Time Alternatives and Exploratory Support Rules

S5.1 Simple serve-time alternatives

Table S5.1 compares serve-time alternatives behind the same screen. Equality checks trade fewer stale admissions for more confirmed-correct declines. The LLM judge declines only

Table S4.7: Exploratory gate mappings and the claim-level equality ablation under the primary-generator replay. Maintenance counts cover 462 request-ticks; endpoint counts cover 154 requests. The core policies and proxy maintenance counts also appear in main-text Section 7.3. Endpoint correct, wrong, and unassessed counts are shown; other quality categories are omitted. Gate scope and alias-update rules differ across configurations: the core write-back policies add retrieval aliases after repair, whereas the final three rows use the default alias-off setting. Every endpoint delivery carries a human label, so no row has an unassessed delivery.

	Maintenance request-ticks				Endpoint requests		
Policy	Accepted regen.	Violating serves	Partially-stale serves	Abstentions	Correct	Wrong	Unassessed
Static cache	3	83	18	18	119	6	0
Blind refresh	421	0	0	41	119	0	0
Page-change loop	174	20	6	36	121	0	0
Strict loop	53	0	0	40	122	0	0
Asserted loop	29	0	18	40	124	0	0
Page-version + overlap proxy	181	0	6	41	121	0	0
Page-version + equality proxy	187	0	0	41	120	0	0
Claim-level equality loop	43	0	0	40	123	0	0

Table S4.8: Crossed gate-by-alias control over 462 request-ticks. Cells report accepted regenerations. The final column is the net count change when alias expansion is enabled. Per-request comparisons confirm exactly two changed decisions, all fallback-to-serve, in each of the asserted and claim-level equality configurations and none in the page-version configurations (`gate_alias_request_diff.json`, `claim_equality_loop_alias.json`).

Gate	Aliases off	Aliases on	On minus off
Asserted claims	31	29	−2
Page-version + overlap	181	181	0
Page-version + equality	187	187	0
Claim-level equality	43	41	−2

6/118 confirmed-correct hits but serves three decayed hits. It assesses the cached question, not incoming-query fidelity, and shares the generation model.

The exact asserted-dependency validator is decision-equivalent to the asserted-claims gate. The latter has the fewest confirmed-correct declines among the reported deterministic conjunctive gates that decline every decayed hit. This is a panel-specific comparison.

The judge requires inference; local gate timings use materialized state and are not an end-to-end latency comparison (Section S7.6). On this uniform-age panel, TTL either expires every entry by the endpoint or leaves similarity-only reuse unchanged.

Alternative certificates: from a negative control to an anchored lexical acceptance rule. The support model permits alternative certificates: reuse is accepted when at least one certificate in a declared family validates (main-text Appendix A.2). Let A_j be the event that branch j validates. Adding a second branch strictly increases acceptance probability exactly when $\Pr(A_2 \setminus A_1) > 0$; independence is unnecessary.

Interpreting that increase as valid certificate availability requires each branch to be accepted under the support and eligibility contract for the current request–response pair. A higher acceptance rate alone does not establish this condition. Joining the asserted-claims primary with a second, sentence-certificate branch under a naive acceptance rule exposes exactly this precondition: the disjunction serves 141/147 hits and declines no confirmed-correct hit—but it serves 8 decayed-or-drifted hits (5 decayed and 3 drifted: $8/147 = 5.4\%$ of similarity hits), six

of them annotated wrong for the incoming request, approaching the similarity-only baseline's 11 decayed-or-drifted serves. In all 8 cases, the recorded sentence strings survive on the target page while the asserted-claims branch fails. The two checks therefore return different availability verdicts. String survival alone does not establish that the sentence supports the current request's answer. The disjunction admits these entries because its sentence branch lacks the primary branch's claim-currency requirement.

The negative control motivates a stricter branch acceptance rule. The *anchored lexical acceptance rule* was formulated after examining failure modes on this panel, so its reported performance is exploratory and in-sample; the rule was then held fixed for the subsequent sensitivity and mechanism checks. It uses current projected claim values as an additional lexical currency check for surviving sentences and does not establish semantic support independently of that representation.

A certificate sentence is *anchored* if it survives on the target page and mentions a value of the property admitted by the current claim projection. An answer-asserted value is *covered* if it is currently admitted or appears in an anchored surviving sentence. The sentence branch qualifies only if it contains at least one sentence, all its sentences survive, at least one is anchored, and every answer-asserted value is covered.

This is a lexical heuristic, not an independently proof-valid branch. A surviving sentence can mention both a stale value and a current value, allowing the stale value to satisfy the coverage rule. Additional admissions beyond the asserted-claims primary are therefore reported as heuristic acceptances, not as a certified route.

Evaluated alone on the 147 similarity hits, the anchored branch accepts 13, all confirmed correct. It rejects all eight decayed-or-drifted hits admitted by the naive disjunction. Joined with the same asserted-claims primary, the family serves 115/147 hits with zero decayed-or-drifted serves and 14 confirmed-correct declines (11.9%; Wilson 95% interval $[7.2, 18.9]\%$; Table S5.1, last two rows). It recovers four confirmed-correct hits declined by the primary alone.

Because the anchored branch strengthens the naive sentence-survival rule, the anchored family is nested within the naive family when both use the same asserted-claims primary. The rule removes 26 naive admissions: eight decayed-or-drifted hits, fourteen confirmed-correct hits, one incomplete hit, and three indeterminate hits. These entries enter fallback rather than being cache-served.

With the shared archived fallback realization, the anchored family delivers 123 correct, 15 incomplete, and two indeterminate answers, with 14 abstentions. It matches the strict policy's observed correct delivery count (Table S4.4) while serving 115 rather than 89 cached answers. This panel-specific outcome does not establish correctness preservation on other workloads.

Outputs are retained in `data/simple_baselines.json` and `data/llm_judge.json` (`s11_simple_baselines.py`). The judge returns 129 YES and ten NO verdicts; eight cases lack evidence and none has a parse failure.

S5.2 Post-hoc exclusions and forward revalidation

The following checks are sensitivity analyses, not independent validation of the anchored rule.

After excluding the development-trail entities—those whose entries were inspected while the anchored rule was being formulated—the retained subset contains 132 similarity hits. The frozen anchored family serves 110 hits, declines 13 confirmed-correct hits, and serves none of the three remaining decayed-or-drifted hits. Because the rule was informed by panel-wide failure analysis, this exclusion does not create an independent safety test.

A per-entry review of the four additional admissions beyond the asserted-claims gate identifies precision changes (Table S5.1): year-level values were replaced by exact dates while surviving sentences continued to state compatible years. All four admissions are annotated

Table S5.1: Serve-time alternatives on the same 147 similarity hits. Reporting groups are eight temporally decayed hits, three meaning-drifted hits, 118 confirmed-correct hits, twelve change-independent incomplete hits, and six indeterminate hits. All mechanisms use the same similarity screen. The exact asserted-dependency validator is decision-equivalent to the asserted-claims gate. The final two rows add a sentence branch using naive string survival or the anchored lexical heuristic, respectively.

serve-time mechanism	served	decayed served	drifted served	correct declined	incompl. declined	indet. declined
page wikitext unchanged	43	1	0	82/118	7/12	5
entity view unchanged	56	0	0	64/118	12/12	4
property claims unchanged	100	0	0	25/118	7/12	4
asserted values in target page	97	3	3	35/118	6/12	4
LLM recheck judge	129	3	0	6/118	1/12	3
PVCacheServe asserted-claims gate	111	0	0	18/118	3/12	4
Exact asserted-dependency validator	111	0	0	18/118	3/12	4
Asserted $\vee$ sentence certificate (naive)	141	5	3	0/118	2/12	1
Asserted $\vee$ anchored sentence heuristic	115	0	0	14/118	3/12	4

correct. This observed case pattern does not eliminate the rule's lexical loophole when stale and current contradictory values co-occur in a surviving sentence.

At the additional claim-state observation on 2026-10-16, 76 days after the 2026-08-01 endpoint, none of the nine decayed entries has a fully restored answer-asserted claim set (surviving asserted claims 0/1, 3/5, 0/1, 0/1, 2/5, 2/4, 1/2, 4/5, 2/3 per entry). At this observation, each entry still fails the conjunctive asserted-claims check. The refetch does not establish the absence of a transient restoration between observations and does not provide a new answer-correctness assessment. No October page snapshot was rebuilt, so the sentence branch cannot be re-evaluated at that date.

S6 Analytic Protection Models and Ablations

We distinguish reliable-copy calculations, static byte-capped placement, and executed maintenance under fallible reads. The first fixes eligibility and assumes reliable backups. The second selects one shared payload set. The executed loop additionally permits backup-read failures, eligibility changes, and unprotected write-back premises. Results from these settings are not interchangeable.

Let P_q be the distinct premises required by a fixed conjunctive contract and T the identities with stored backup payloads. Placement alone does not establish eligibility. Conditional on all required premises being eligible, independent source-read failures at rate p and reliable backups give

$$R_q(T) = (1-p)^{|P_q \setminus T|}.$$

If source and backup reads instead fail independently at rate p, main-text Corollary 4.2 gives

$$R_q(T) = (1-p)^{|P_q \setminus T|}(1-p^2)^{|P_q \cap T|}.$$

Repeated references count once. An ineligible required premise forces rejection regardless of placement. These are fixed-state formulas, not whole-trajectory models.

S6.1 Static recovery under read loss

The static recovery overlay uses the strict and asserted-claims contracts on the same 147 similarity hits. Each required item's source read fails independently with probability p; a

Table S6.1: Expected served hits under independent read loss, out of 147 similarity hits. Model R independently protects each unique premise with probability c. Model F protects $x = \lfloor 0.9n \rfloor$ required premises per certificate. Intact-read baselines are 111 asserted and 89 strict serves. Full reliable protection reproduces those verdicts. The reported analytic expectations agree with the seeded Model R Monte Carlo check to within 0.03 serves.

contract	p	no protection	R: c=0.90	R: c=0.95	F: x=$\lfloor 0.9n \rfloor$	full
asserted	0.01	109.3	110.8	110.9	109.9	111
	0.05	102.9	110.1	110.6	105.4	111
	0.10	95.6	109.3	110.1	99.7	111
	0.20	82.1	107.6	109.3	88.5	111
strict	0.01	85.2	88.6	88.8	88.1	89
	0.05	71.7	87.1	88.0	84.3	89
	0.10	58.0	85.2	87.1	79.7	89
	0.20	37.7	81.5	85.2	70.4	89

reliable protected copy repairs that read only if the item remains eligible. Both contracts are conjunctive, so any required item left unreadable causes a decline. For a fixed eligible certificate, this is the leaf-only case of main-text Theorem 4.1. We evaluate two protection models (formulas, budget accounting, and ablations in Section S6.2): *independent placement* (model R) and *per-certificate packet budget* (model F). Without protection at $p = 0.10$, the asserted-claims contract retains an expected 95.6 of its 111 intact-read serves, whereas the strict contract retains 58.0 of 89. Their mean required-set sizes among intact-read serves are 1.59 and 4.43, respectively. Larger required sets therefore impose an additional read-availability cost beyond the intact-state declines reported above.

The separate global byte-budget comparison uses one coverage-frequency placement under a common cap. This is a payload-coverage comparison, not a claim of current eligibility or gate acceptance. Section S6.2 reports the coverage, allocation, and sharing ablations.

With reliable backups and $0 < p < 1$, a fixed eligible certificate has no read-loss penalty exactly when every required premise is protected. Protecting the entire placement universe is unnecessary. Eligibility must still be checked: the copy-trust negative control serves all 147 hits, including all eight decayed hits, by trusting restored base-period bytes without the currency audit. That control violates the restoration contract. The executed sweep separately probes protected ineligible payloads and rejects matching bytes before injected read faults are evaluated (Section S7).

The forward eligibility observation of Section S5.2 shows that partial claim restoration cannot satisfy the conjunctive gate.

Extended executed-loop diagnostics are reported in Section S7; headline regeneration counts appear in main-text Table 8.

S6.2 Model derivations and placement ablations

This section gives the derivations and ablations behind Table S6.1. The expected serve counts sum read-survival probabilities over entries that pass the corresponding intact-read contract. Entries that fail eligibility remain declined under both protection models. Here n is the number of distinct premises required by that entry's contract.

For *independent placement* (model R), each unique premise receives an independent placement indicator with probability c. An item is unavailable only when it is unprotected and its source read fails, giving certificate survival $(1 - p(1 - c))^n$, averaged over placement and loss.

For the *per-certificate packet budget* (model F), a fixed subset of $x = \lfloor 0.9n \rfloor$ items is protected, giving survival $(1 - p)^{n-x}$. These are applications of the residual-premise law in main-text Theorem 4.1.

The numerical difference between R and F should not be attributed to randomization alone. Their protected counts differ: for $n = 1$, model R protects the item with probability 0.9, whereas model F protects none. At $p = 0.10$, their expected asserted-claims serves are 109.3 and 99.7, respectively; the strict-contract values are 85.2 and 79.7. We therefore compare placements separately under a common global byte cap.

Placement comparison. The residual law determines expected serves for a specified placement, not from a byte cap alone. The coverage-frequency heuristic omits the footprint-dependent marginal gains in the main-text placement objective and does not rank benefit per payload byte. The fixed placement ranks payload objects by certificate-reference frequency and greedily adds items that fit the remaining budget. The random comparator draws 50 feasible sets: each proposal includes every unique premise independently with probability equal to the fixed placement's realized byte share and is rejected if its payload bytes exceed the cap; each accepted placement is held fixed across 100 failure realizations. An unconditioned Bernoulli analytic reference at the corresponding nominal expected byte share is also retained; its expected serve counts differ from the feasible-set results by at most 0.8.

At a 90% budget, the fixed placement covers all designated premises of 115 of the 147 entries. For an eligible certificate, complete coverage removes source-read exposure when backups are reliable. The fixed placement has higher serve counts than the random comparator at the reported 90% and 95% budgets. The ordering reverses in the reported 50%-budget strict comparison: 77.4 versus 79.7 expected serves at $p = 0.05$.

The storage representation has 782 references to 720 payload objects, a ratio of 1.09. Of these objects, 677 occur once; multiply referenced objects account for 2.9% of bytes. These are storage-key statistics, not counts of shared context-specific claim identities. The per-citation ablation changes reported serve counts by at most 2.7. Sharing alone therefore does not explain the high-budget ordering.

These comparisons evaluate retained placements, not optimal allocation. Used bytes were not retained for the rejection-sampled random sets, so allocation structure and utilization cannot be separated. Model F is a per-certificate reference; the global experiment uses one shared placement.

The overlay outputs are retained in `data/protection_recovery.json`. The 2026-10-16 claim refetches are retained under `data/current_claims/`; their interpretation appears in Section S5.2.

S7 Implementation and Extended Execution Audits

This section reports certificate handoff, executed restoration, paired trajectory diagnostics, partial-label quality accounting, and local timings. Headline results appear in main-text Section 7. The class-aware synthetic policy accounting is reported separately in Section S1.

S7.1 Transferable certificates and verification boundary

The separate handoff audit serializes each collected support record into a checksum-bound envelope containing the request identifier, answer digest, recorded asserted and designated claim sets, sentence identifiers, and issuance snapshot. A canonical-serialization hash binds these fields. The verifier supplies the audit snapshot separately and evaluates the selected contract.

This handoff assumes an honest issuer. The unkeyed seal detects tested changes made without updating the seal; it does not authenticate an envelope against an adversary who can rewrite both content and digest. Authenticated deployment would additionally require a keyed tag or signature and a bound contract-version identifier. Successful envelope verification also

does not establish that the issuer selected semantically sufficient premises or that the verifier's current-state provider is accurate. The handoff audit checks binding and verdict consistency within the declared trust boundary.

S7.2 Transferred-envelope audit

This separate mechanism audit uses an earlier replay with template fallback at intermediate ticks. It transfers 183 envelopes: 154 base records and 29 write-back records. These are not the response-conditioned records of the headline real-output trajectories. A separately implemented verifier agrees with the in-memory checker on all 2,928 envelope–snapshot–contract combinations ($183 \times 4 \times 4$).

Six modification classes are rejected for every envelope: answer substitution, designated-claim modification, asserted-claim modification, sentence substitution, issuance-date modification, and cross-request replay. A further 86 claim-trimming attempts on 43 endpoint-ineligible base records are rejected.

All modifications retain the original seal. The audit checks field binding and verdict consistency under an honest issuer; it does not evaluate adversarial authentication.

S7.3 Protection in the executed closed loop

We execute reuse-path read failures in the strict and asserted-claims loops of main-text Section 7.3. The executable read chain uses local SQLite payload stores and the restoration procedure of main-text Section 5.4. Request-local source-read faults use the stable keys defined in main-text Section 6.5. A failed source read of a protected item invokes the backup-read, fingerprint, and eligibility chain. Repeated references to one premise within a certificate denote one required evidence item, not independent copies. The data exercise two payload-intact failure cases without injected flags. A protected claim can retain matching bytes while its value is absent from the current claim projection; a protected sentence can retain matching bytes while its string is absent from the current page. Both objects verify their fingerprints and are still declined: the checker rejects them as contract-ineligible and version-incompatible, respectively. No policy-permission revocation is tested.

The SQLite chain stores base claim-value keys and sentence text. Post-build objects outside the base universe use placeholder rows and are not protected by budgeted placement. Their availability is evaluated through the snapshot registry and fault model. The experiment executes backup reads for selected base objects; it does not benchmark end-to-end reconstruction of new evidence payloads.

At $p = 0.10$, the fixed arm records mean claim-ineligibility guard counts of 49.3 for asserted reuse and 56.5 for strict reuse, plus 14.0 sentence-version guard events under strict reuse. These probes read matching protected payloads and reject them before the injected read-loss branch. They count guard evaluations, not distinct premises or loss-triggered restoration attempts.

Budgeted arms independently lose source and backup reads at rate p; the ideal read-any reference removes read loss while retaining eligibility checks. Placement uses base-period information only (main-text Section 5.4).

Main-text Table 8 reports accepted regenerations over three update ticks, averaged over 200 coupled fault realizations per cell. Every reachable fallback key has an archived model output or an explicit abstention, with coverage checked programmatically (protocol s36/s37; main-text Appendix D). The erasure-free real-output-driven loops use 29 accepted regenerations for asserted claims and 53 for strict reuse. Write-back premises are extracted from actual accepted answers. Earlier template-driven trajectories are separate mechanism checks and are not pooled with these results.

In the unprotected strict arm, 65.8 accepted regenerations at $p = 0.05$ and 109.9 at $p = 0.10$ follow a read-loss-induced decline of a certificate that passes the intact-read check at the same

snapshot. This diagnostic holds the certificate and snapshot fixed; it is distinct from cross-arm trajectory pairing. It measures contract availability, not human-labeled false declines. These counts can exceed the arm's excess over the erasure-free reference ($115.7 - 53 = 62.7$ and $158.0 - 53 = 105.0$). The remaining accepted-regeneration counts ($115.7 - 65.8 = 49.9$ and $158.0 - 109.9 = 48.1$) fall below the erasure-free count of 53. These differences arise between evolving write-back trajectories and do not identify which earlier repair, if any, prevented a later reference regeneration.

At $p = 0.10$, the executed shuffled-fill comparator records 37.4 and 81.2 accepted regenerations for asserted and strict reuse. It uses the same byte budget as fixed placement and differs from the rejection-sampled static comparator of Section S6.2.

The residual decomposition (protocol s38; main-text Appendix D) concerns asserted reuse with fixed protection at $p = 0.10$. An accepted regeneration is assigned first to the reference-nonserve category when the matched erasure-free request-tick either regenerates or abstains. Remaining events are assigned to protected double-read loss, then unprotected or write-back loss.

The respective means are 28.5, 5.4, and 0.3 per run, against 34.2 accepted regenerations in total. Rounded components need not sum to the rounded total. This is a precedence-based accounting partition, not a causal decomposition: write-back histories differ, and reference non-serve does not establish that evidence change alone caused regeneration.

S7.4 Paired attribution and maintenance costs

At p=0.10, paired asserted-contract runs cover 92,400 request-ticks (protocol s39; main-text Appendix D). Let n_{ab} count transitions from unprotected decision a to protected decision b, with $a, b \in \{S, F, A\}$ denoting cached serve, accepted fallback, and abstention. Then

$$N_F^U - N_F^P = n_{FS} + n_{FA} - n_{SF} - n_{AF}.$$

Here $n_{FS} = 7{,}823$, $n_{SF} = 942$, and $n_{FA} = n_{AF} = 0$, giving $6{,}881/200 = 34.405$ fewer accepted regenerations per run.

The arms share exogenous faults but evolve separate write-back states. These transitions compare trajectories, not isolated protection effects at identical cache states.

The local payload-chain microbenchmark reports 11.3 ms for registry construction, 1.4 ms for loading the four snapshot registries in total, 0.5 μs per gate check, and 4.9 μs per backup read. Its 183 certificate records occupy 30,878 logical bytes. These are component measurements, separate from the real-output routing sweep.

At $p = 0.10$, the asserted fixed arm records 10,688 restorations and 9,856 ineligibility probes over 200 runs. Both paths perform a backup read, giving 102.72 reads per run. Applying the separate 4.9 μs component timing gives approximately 0.50 ms per run, or 15 μs per avoided accepted regeneration. This estimate includes probe reads but excludes fingerprint and eligibility checks, placement, upkeep, and remote access. It is not an end-to-end cost or break-even estimate. The headline sweep comprises 4,800 fault realizations across two contracts, four protection settings, three read-loss rates, and 200 realizations per cell. These repeat the same workload under different fault seeds; they are not independent semantic test cases. No cached delivery in the headline real-output-driven sweep satisfies the violating-serve condition $A \not\subseteq L_t$ defined in main-text Section 6.3. This follows from the implemented asserted-claim eligibility check; partially-stale serves remain a separate category. Earlier template-driven executions are mechanism checks and are not pooled with these trajectories.

S7.5 Fault-run quality under incomplete annotation

Fault-run quality uses the label-aware fields in `real_driven_loop.json`, with unlabeled outputs counted as unassessed. The similarly named endpoint-quality fields in

`real_driven_sweep.json` are not used for human-labeled quality claims. The sweep file supplies routing totals, p95 values, and guard counts.

The erasure-free endpoint comparison in main-text Table 7 has complete delivery labels. Fault runs can instead reach archived gap-fill outputs without human correctness labels. These deliveries are recorded as unassessed and are not counted as correct. Confirmed-correct counts are therefore lower bounds under the retained annotation, not complete estimates of correct delivery.

At $p = 0.05$, 0.10, and 0.20, unprotected confirmed-correct endpoint counts are 112.8, 103.0, and 86.4 for the asserted contract, compared with its erasure-free count of 124. The strict-contract counts are 100.6, 89.2, and 79.9, compared with 122. Fixed 90%-byte protection limits the corresponding losses to 0.5–9.1 and 2.8–21.4 answers; the ideal read-any reference preserves the erasure-free counts.

For the asserted contract, unprotected runs have 13.6, 24.4, and 42.9 unassessed endpoint deliveries at the three loss rates. Fixed protection has 0.7, 3.0, and 11.1. Because annotation coverage differs across arms, differences in confirmed-correct counts combine delivery changes with assessment-coverage differences. They do not, on their own, estimate the difference in total correct delivery.

Protocol s30/s37/s35; outputs indexed in main-text Appendix D.

S7.6 Staged timing and fail-closed behavior

Local serve-path timing. The post-embedding path is measured on the same Apple M3 Pro over 3,000 randomized hit evaluations with pre-materialized target evidence. Stage medians are 6.1 μs for exact brute-force retrieval over 154 cached embeddings, 0.1 μs for certificate access, 0.4 μs for claim checks, and 1.5 μs for sentence checks.

Per-request totals are timed separately; stage medians are not summed. The strict-contract total has median 8.2 μs and p99 12.7 μs. Claims-level contracts omit sentence checks and have median 6.7 μs. Retrieval dominates at this cache size. Query embedding is excluded: it is common to the compared cache policies but remains part of end-to-end latency.

Retrieval scaling. Under the same brute-force implementation and embedding dimension, synthetic-cache measurements are 18 μs at 10^3 entries, 455 μs at 10^4, 4.4 ms at 10^5, and 45 ms at 10^6. Larger deployments may therefore require indexed retrieval, with the choice governed by latency and recall requirements. Evidence checking takes microseconds in this local measurement with fixed certificate sizes and an in-memory registry. The experiment does not establish verification latency across larger registries, different access patterns, or remote state providers.

Injected faults. The injected-fault audit evaluates all 147 similarity hits. With entity evidence unavailable, corrupted claim reads, or an unreadable certificate, every contract serves 0/147. With page evidence unavailable, strict and majority-sentence contracts serve 0/147 in this audited workload; claims-level contracts are unaffected because they do not require page sentences. A corrupted query embedding produces a retrieval miss. The no-fault control reproduces the baseline serve counts 89, 96, 104, and 111. These are outcomes for the tested faults and certificates, not a claim that every individual sentence-read failure invalidates a majority contract.

The timing and injected-fault outputs (`s14_staged_timing_failclosed.py`) are retained in `data/staged_timing_failclosed.json`.

S7.7 Retrieval records and local timing

Retrieval-record protocol. The corpus retains the first HotpotQA context passage encountered for each title. Each record is embedded from its title followed by up to 400 characters of

the leading passage fragment, obtained by splitting the stored text at the first `". "` delimiter. Queries retrieve records by cosine similarity using the same encoder. Premise records hash the stored dataset passage and attach its source identity and fixed version, policy, and index metadata.

Timing environment. A single-process microbenchmark uses `time.perf_counter_ns` on an Apple M3 Pro (arm64, 18 GB, CPython 3.12). Its workload comprises the supplied dataset contexts for the 7,405 HotpotQA questions, not the retrieved records used for the availability curves. Each question supplies a full-context set of at most ten records (mean 9.95) and a gold-title support subset (mean 2.0).

Record construction builds both record sets in one pass, including SHA-256 hashing of the stored passage, version-tag assignment, and an index-membership probe. It is timed per question: median 9.0 μs, p95 12.0 μs, p99 14.1 μs. *Checking* is timed per call, pooling nonempty support subsets and full-context sets. No warm-up pass is used. The intact arm repeats passes until 20,000 calls are available. The erasure arm draws a fresh erasure mask for every call across repeated passes (twenty at most) and stops once 20,000 pooled calls have been timed. Both arms therefore report percentiles over 20,000 retained pooled calls, using linear interpolation between adjacent order statistics. Erasure-mask generation is excluded from checker timings. Median and p99 costs are 0.67 and 1.9 μs for intact sets, and 0.42 and 1.7 μs under $\varepsilon = 0.1$ erasure with early exit. Per-call intervals exclude loop bookkeeping. The pooled checker percentiles combine support-subset and full-context calls; they are not percentiles for a single deployed gate workload.

Erasure masks determine presence; policy checks use fixed allow values, and hash comparisons consult a memory-resident registry. The timings exclude remote probes, source rereading, current-hash recomputation, invalidation maintenance, and end-to-end serving. Figure S1.1(c) uses the erasure-sampled checking series.

S8 Daily Service-Demand Scenario

This section converts daily routing counts into nominal service demand. It is a scenario, not realized API accounting.

Cost model. For policy p with fallback share ρ_p/N,

$$C_{1000}(p) = 1000\Big[c_{\text{gate},p} + c_{\text{state},p} + \tfrac{\rho_p}{N}\, c_{\text{gen}}\Big].$$

Gate costs are 9.8 μs (asserted) and 11.4 μs (strict). The generation reference c_{gen} is the upper middle of six question-only API timings: 1.12 s (Kimi) and 3.82 s (GPT). These probes omit fallback evidence, retries, and acceptance failures. The source re-read cost is the median of three live page fetches, 1.775 s (`stage6_load_cost.py`, `load_cost.json`).

Access conventions. Under maintained state, only the page-hash and infobox-diff references are charged a per-case re-read; other state-dependent arms read a maintained audit state whose upkeep is excluded. Under the re-read convention, every state-dependent arm is charged $c_{\text{state}} = 1.775$ s. TTL arms require no state in either convention.

Under the re-read convention, TTL-1d has lower nominal demand than the asserted gate for Kimi but not for GPT, while still issuing about five times as many fallback invocations. The comparison independent of the access convention is fallback demand.

Table S8.1: Nominal seconds per 1,000 answer–tick cases (Kimi/GPT) under the two access conventions.

arm	maintained state	re-read for all state-dependent arms
TTL-1d	1,062/3,629	1,062/3,629
TTL-14d	335/1,146	335/1,146
page-hash (ref.)	2,683/4,901	2,683/4,901
GC/FC mappings	908/3,126	2,683/4,901
infobox-diff (ref.)	2,423/4,050	2,423/4,050
sent-asserted	208/753	1,983/2,528
sent-strict	242/865	2,017/2,640
KV-claims	48/149	1,823/1,924

S9 Daily-Slice Replay

This section specifies the daily dual-generator panel and its quality projections. Section S9.1 gives construction, routing, and resampling; Section S9.2 gives error accounting. A superseded single-generator pilot is retained in the artifact under `experiments/15_second_domain_daily_replay/` and contributes no reported result.

S9.1 The dual-generator main panel

Expanded pool and claim anchoring. The pool is expanded to 100 candidate pages in five groups (54 World Cup players, 16 national teams, 14 tournament and season pages, 6 weather-season pages, 10 stable controls); 12 pages with zero in-window edits are filled with their exact t_0 boundary revision and flagged `static_in_window`, and one redirect is removed, leaving 99 usable pages (`stage1c_fetch_expanded.py`, `stage1d_static_fill.py`). Claim anchoring adds keyword and proximity constraints to normalized matching (`stage2e_build_panel_prox.py`). For numeric key stems with a configured topic mapping, support requires the mapped keyword and an eight-token proximity check. Unmapped numeric stems use the value-matching rule without that topic constraint. Mapped textual role keys require a topic keyword in the support sentence. After content filtering (1,044 candidate pairs dropped) and on-topic support screening (1,920 dropped), 373 eligible claims form the pool; three same-page claims compose one answer, giving 99 answers over 297 claims, all included without sampling (29 pages contribute no claim and 13 fewer than three). Within each page, claims are ordered by infobox key and grouped into non-overlapping consecutive triples; a per-page remainder of one or two claims is not used, so 76 of the 373 eligible claims do not enter any answer. All 99 constructed answers are retained: six have at least one claim superseded at the final tick and 93 do not. No answer subsampling or weighting is applied. Pool-level supersession rises from 0.0% at one day to 0.54%, 1.07%, 2.95%, and 2.95% at three, seven, fourteen, and twenty days (`pool_stats21.json`).

Question templating and dual generation. Each answer becomes one three-part question: one sub-question per claim from key-stem templates (e.g., `currentclub` → "Which club does P currently play for?"; `nationalcaps` → "How many appearances (caps) does P have for their national team?"), with the generic fallback "What is the *key* of P, as listed in its Wikipedia infobox?"; the prompt supplies the certificate sentences as evidence and requests numbered concise answers as of t_0 (`stage0_generate_answers.py`). Kimi (`kimi-k2-turbo-preview`) and GPT (`gpt-5-mini` via the Poe API, OpenAI-compatible) each answered all 99 questions; lexical value-mention acceptance retained 62 and 68 entries, respectively (`answers_kimi.json`, `answers_gpt.json`). Acceptance is set-level. For textual values, the normalized value must occur somewhere in the answer. For values containing digits, every extracted numeric sub-

string must occur, without token-boundary checks. Normalization ignores case, commas, and whitespace. The check does not verify part assignment or the answer's remaining assertions. Fallback uses the same normalization and all-match rule on the current infobox values associated with the selected keys. A key absent from the current snapshot contributes no matching requirement and is skipped rather than failed (`stage5_fallback_regen.py`). Thus acceptance does not verify the answer's treatment of a missing value.

Sentence-contract predicates. For cached entry j, let $\mathcal{S}_j^a$ be the set of distinct stored support-sentence identities required by the asserted sentence contract, and let $\mathcal{S}_j^c$ be the additional context identity required by the strict contract. For each identity s, let $V_k(s)$ indicate survival at tick k under the daily normalized-text predicate: the normalized sentence (case, comma, dash, and whitespace normalized) appears as a substring of the normalized current lead. The two acceptance rules are

$$G_j^a(k) = \bigwedge_{s \in \mathcal{S}_j^a} V_k(s), \qquad G_j^s(k) = \bigwedge_{s \in \mathcal{S}_j^a \cup \mathcal{S}_j^c} V_k(s).$$

The construction is as follows (`stage2e_build_panel_prox.py`). For each of the three claims of entry j, the stored support is the *first* t_0 lead sentence that contains the claim's declared infobox value under the topic-proximity matching of Section S9.1; because claims can share a sentence, $\mathcal{S}_j^a$ may hold fewer than three identities (this happens for 75 of the 99 panel answers), and repeated references count once. The set $\mathcal{S}_j^c$ contains at most one identity. Context is selected once per page: the first base lead sentence containing none of that page's eligible claim values. Its identity is shared by the page's three-claim entries. If no such sentence exists, the context set is empty and the strict gate reduces to the asserted sentence gate. Here "asserted" names a sentence-level support scope. It is not the monthly asserted-claims predicate of Section S4, which checks lexically extracted claim identities against the projected eligible-claim view: the daily checker tests normalized textual persistence, not sentence entailment or current factual correctness.

Daily-scenario arm definitions. All arms share the maintained tick snapshots, the dual caches, and the regeneration accounting of the dual replay (`stage4_dual_replay.py`); the mechanism arms port the Section S4.5 mapping conventions to this scenario's signals (`stage7_mechanism_arms21.py`):

- KV-claims: serve if and only if all three declared premise keys still hold their t_0 infobox values (no page-version guard).
- GC-overlap (GroundedCache-style): serve if and only if the page hash is unchanged, at least two of the three claims remain valid, and every asserted t_0 value still occurs in the current lead.
- FC-equality (FinCacheServe-style): serve if and only if the page hash is unchanged and all three claims remain valid.
- page-hash: serve if and only if the page revision hash is unchanged (source re-read reference).
- infobox-diff: serve if and only if no infobox key changed (source re-read reference).

FC-equality combines the three-key validity condition with a page-version guard. GC-overlap instead combines that guard with an at-least-two claim-validity condition and asserted-value presence in the lead. Its predicate is therefore not KV-claims plus an additional guard.

On the retained cases, the extra conditions of both mapped arms are satisfied whenever the page-version guard passes. Their routing consequently matches the page-hash reference. This is an observed relationship on the panel, not a general equivalence of the predicates.

Frozen-cache accounting. The replay retains the base answer and certificate without write-back. Each answer–tick case is evaluated against the current tick state, and each rejection contributes one fallback invocation to the reported demand; repeated rejection of one entry is counted repeatedly. A later restoration of the base premises restores acceptance: on this panel, 4 of 62 Kimi and 5 of 68 GPT cached entries pass the asserted gate again after an earlier failure (across the full 99-answer panel, 14 claim-level sentence survivals and 5 infobox-value assignments revert from failed to valid within the window). Non-recovery is therefore not imposed by the frozen-cache definition, and the routing counts already reflect per-tick re-evaluation.

Under the asserted sentence contract, 19 of 62 Kimi entries and 22 of 68 GPT entries are rejected at least once. Real generation is executed at those 41 first-rejection cases. The full frozen routing table counts demand at all evaluation ticks, not a complete set of executed regeneration calls.

Evaluation ticks and TTL boundary convention. The 21 snapshots comprise the construction state t_0 and 20 subsequent evaluation states. Thus the retained caches contribute $62 \times 20 = 1{,}240$ Kimi cases and $68 \times 20 = 1{,}360$ GPT cases. The frozen-cache TTL rule rejects an entry only when its age is strictly greater than the stated duration:

$$\mathrm{Reject}_d(k) = \mathbf{1}[k > d], \qquad k = 1, \ldots, 20.$$

Consequently, TTL-1d, 3d, 7d, and 14d reject 19, 17, 13, and 6 cases per entry, respectively. These are boundary conventions for the frozen replay, not an implementation-independent definition of TTL. No refreshed answer is written back in this scenario.

Fallback regeneration and load measurement. `stage5_fallback_regen.py` regenerates each cached answer at its first asserted-gate rejection with the same generator and the current lead as evidence, judging against tick-k truth (`fallback_regen.json`: lexical value matching accepts Kimi 17/19, GPT 20/22). `stage8_spotcheck_sheet.py` draws the seed-42 sample of 20 regenerations (11 Kimi, 9 GPT) and emits the double-annotation sheets, which withhold the automatic judgments; `stage9_parse_spotcheck.py` parses the two returned sheets, audits agreement, and adjudicates conservatively (`fallback_spotcheck_results.json`; annotated sheets under `annotation/`). The two annotators agree on 19/20 cases (Cohen's $\kappa = 0.90$); the single disagreement (an all-hosts listing against per-part country targets) is adjudicated incorrect under the conservative rule. Adjudicated correctness is 9/20 (45.0%, Wilson 95% [25.8, 65.8]), against the lexical acceptance of 19/20 in the sample: value matching over-accepts, with 10 false accepts dominated by per-part mismatches masked by cross-part mention (7), a value asserted where the infobox lists none (1), "not listed" answers where values exist (1), and per-part over-listing (1); its single rejection (year convention) was confirmed incorrect by both annotators. Cost references for the service-demand scenario are described in Section S8. The spot-check sample is drawn from first-rejection regenerations, which include regenerations of falsely rejected valid cases; its estimate concerns three-part regenerations under set-level lexical acceptance and is not an estimate of cached-answer quality. The spot-check labels count wrong and incomplete answers alike as incorrect; they are therefore used with the same error definition as the base-answer sample of Section S9.2.

Page-cluster bootstrap. Policy contrasts use 10,000 paired page-cluster resamples (seed 42): 42 Kimi pages and 47 GPT pages (`stage13_pagecluster_bootstrap.py`). Each drawn page contributes all of its accepted answers and evaluation ticks; a page drawn repeatedly contributes them repeatedly, so bootstrap multiplicity is preserved. Rates are ratios of summed counts, not averages of page rates.

For asserted-minus-TTL-14d stale delivery, intervals are $[-7.36, -0.46]$ percentage points for Kimi and $[-6.81, -0.41]$ for GPT. For fallback demand against TTL-1d, they are $[-86.4, -65.2]$ and $[-84.3, -64.8]$. Against GC-overlap, they are $[-74.3, -49.6]$ and $[-72.8, -50.2]$.

The cross-generator comparison pairs the 41 shared pages and gives $[-0.01, +0.03]$ percentage points for asserted-gate stale delivery. It is page-matched, not question-matched: accepted question sets can differ within a shared page. The interval establishes neither a generator difference nor equivalence.

A served-conditional rate is undefined with zero serves. A separate seed-42 denominator audit observed no zero-denominator draws in the full or filtered analyses (`stage15_zero_denominator_check.py`). Outputs are retained in `pagecluster_bootstrap.json` and `zero_denominator_check.json`.

S9.2 End-to-end error accounting

Observed routing versus projected answer error. The replay directly observes contract decisions and source-state-defined staleness. The end-to-end error tables are not a complete human annotation of all answer-tick deliveries. They combine those routing counts with quality estimates from sampled base answers and first-rejection regenerations. The projection assumes that every source-defined stale cached delivery counts as an answer error and that the quality parameters transport to the cases rejected by each compared policy. Source-defined staleness alone does not establish that an answer asserts the changed value. Accordingly, the tables evaluate the declared quality model rather than directly measuring the actual error rate of every strategy. The monthly one-shot analysis differs: its rejected-set identity uses answer labels and shared archived fallback outputs directly.

Main-text Proposition 5.2 gives the end-to-end comparison. With N answer-tick cases, s stale cases, ρ rejected cases of which ρ_s are stale, $m = s - \rho_s$, fallback acceptance a, and error probabilities r (delivered fallback) and b (non-stale cached), where errors are wrong or incomplete answers,

$$E_{\text{pol}} - E_{\text{sim}} = \rho\,(ar - b) - \rho_s(1 - b).$$

For the asserted gate (Kimi: $N = 1{,}240$, $s = 53$, $\rho = 231$, $\rho_s = 52$, $m = 1$; GPT: $N = 1{,}360$, $\rho = 268$, $\rho_s = 52$), the precision is $\pi = 0.225$ and 0.194, and the gate has fewer errors than similarity-only reuse if and only if $(ar - b)/(1 - b) < \pi$. Against TTL-1d (Kimi $\rho' = 1{,}178$) it has fewer errors if and only if $ar - b > (1 - b)/947 \approx 0.0011\,(1 - b)$. Table S9.1 lists the break-even threshold $\pi + (1 - \pi)\,b$: strict improvement requires ar below this threshold, and equality gives the same expected error count.

Table S9.1: Break-even fallback error mass ar (%) relative to similarity-only reuse under the homogeneous model, by cached-answer error rate b. Strict improvement requires $ar < \pi + (1 - \pi)\,b$; the entries are thresholds, not observed fallback-quality estimates. π is the policy's precision. Calibrated values: $b = 22.8\%$; $ar = 50\%$ (set-level) and 15% (per-part re-scoring).

policy	π	b=0	10	20	22.8	30	45
KV-claims (both)	1.000	100	100	100	100	100	100
asserted, Kimi	0.225	22.5	30.3	38.0	40.2	45.8	57.4
asserted, GPT	0.194	19.4	27.5	35.5	37.8	43.6	55.7
page-hash, Kimi	0.053	5.3	14.7	24.2	26.9	33.7	47.9
TTL-1d, Kimi	0.045	4.5	14.0	23.6	26.3	33.2	47.5

Calibration. Both r and b count wrong or incomplete answers as errors. In the 20-case fallback spot check, set-level matching accepts 19 and 10 of these are adjudicated incorrect; the rejected case abstains, so $a = 19/20$, $r = 10/19$, and $ar = 0.50$ (Wilson

[29.9, 70.1]). A stratified 40-answer sample of accepted base answers (10 per-part-flagged and 10 clean per generator; `stage11_base_answer_sheet.py`, `base_answer_sample40.json`) was double-annotated under the spot-check protocol (`stage12_parse_base_annotations.py`, `base_answer_results.json`). The annotators agreed on all 40 items ($\kappa = 1.0$): 22 correct, 14 incomplete, 4 wrong, none indeterminate. Flagged items are non-correct in 18/20 cases and clean items in 0/20. The pooled flagged-stratum calculation, with the flagged share 33/130 as design weight, gives

$$\widehat{b} = \frac{33}{130} \cdot \frac{18}{20} \approx 22.8\%.$$

Generator-specific weighting gives 23.2% for Kimi and 22.5% for GPT and leaves the reported point-estimate comparisons unchanged. The primary pooled calculation uses $b = 22.8\%$. The error budgets use the rounded value $b = 0.228$; the unrounded estimate 0.2285 changes no reported entry by more than 0.5 per 1,000 cases and no verdict. Applying the design weights to the endpoints of the two marginal Wilson intervals gives an envelope of $[17.7, 36.7]\%$. This is a sensitivity envelope constructed from stratum-wise nominal 95% intervals, not a confidence interval with a proved 95% simultaneous coverage guarantee for the weighted total; the clean stratum's zero observed errors remain a source of uncertainty rather than evidence of zero population error.

Transport assumptions. The daily error projection makes the following assumptions.

1. The sampled first-rejection regenerations represent fallback quality on repeated rejections and on the rejection sets of the other policies.

2. Pooled Kimi and GPT fallback assessments can be used for each generator-specific routing stream.

3. The base-answer sample estimates error among non-stale cached deliveries at the evaluated ticks.

4. Policy-specific differences in rejected-case difficulty do not materially change a, r, or b.

5. The transported per-part budgets retain the original routing counts and precision after changing base-cache eligibility. A separate restricted-cache replay below measures the resulting routing changes directly. It does not establish that precision is unchanged or validate the remaining fallback-quality transport assumptions. Requests whose base entries are removed are outside the restricted-cache denominator; their service cost is not included.

These assumptions are not consequences of certificate soundness. The first-rejection design does not independently estimate policy-, tick-, or difficulty-specific fallback quality. The posterior calculations quantify uncertainty within this transport model, not uncertainty about the validity of the model itself.

Set-level acceptance. At $b = 22.8\%$ and $ar = 0.50$, the asserted gate lies outside its improvement window: $ar - b \approx 27$ percentage points exceeds $\pi(1 - b)$, which is 17.4 points for Kimi and 15.0 for GPT. Its point-estimate error remains below TTL-1d. The marginal sensitivity ranges overlap and do not resolve the sign of $ar - b$ or improvement over similarity-only reuse. Table S9.4 summarizes joint uncertainty under the stated posterior model. Table S9.3 gives the GPT budget.

Per-part acceptance. Per-part re-scoring parses numbered `(1)`, `(2)`, and `(3)` segments and checks each value in its corresponding segment. The implementation falls back to whole-answer matching when too few segments are parsed. No retained base or fallback output is marked unparsed in this analysis. Missing current values are skipped, as under set-level acceptance. Re-scoring without new model calls accepts 24/41 fallback outputs, versus 37/41 under set-level matching (`stage10_perpart_rescore.py`, `perpart_rescore.json`). In the spot-check sample, it rejects seven of ten erroneous accepted outputs and one correct output, while retaining the original year-convention rejection. The resulting 11 accepted outputs contain eight correct and three incorrect answers: $a = 11/20$, $ar = 3/20 = 0.15$ (Wilson interval $[5.2, 36.0]\%$).

Per-part matching flags 16/62 Kimi and 17/68 GPT base entries. In the base audit it flags all 18 non-correct answers and two correct answers. The clean stratum has 0/20 observed errors, with Wilson upper bound 16.1%.

Table S9.2 transports the original routing counts to this acceptance rule. At $b = 0$, KV-claims and the asserted gate improve on similarity-only reuse; page-hash and TTL-1d do not. At $b = 16.1\%$, $ar < b$, and TTL-1d has the lowest error among the listed policies. These budgets do not charge the coverage loss from removing base entries. The separate restricted-cache replay below measures changed routing but does not validate fallback-quality transport.

Per-part-filtered replay. We separately re-run routing after restricting the accepted base caches to entries retained by per-part matching: 46/62 Kimi entries and 51/68 GPT entries (`stage14_perpart_filtered_replay.py`, `perpart_filtered_replay.json`; no new model calls). The corresponding analysis populations contain 920 and 1,020 answer–tick cases.

The asserted sentence gate has fallback demand of 16.7% and 17.1%, with directly measured rejection precision of 23.4% and 25.9%. Both fallback-demand shares are below the original values, while precision is above the original 22.5% and 19.4%. The served-conditional stale rates are 0.13% and 0.12%, slightly higher than in the original caches because the analysis population and serving denominator have changed. Filtering therefore does not improve every routing metric.

The reported paired page-cluster intervals preserve the stale-delivery contrast with TTL-14d: $[-9.4, -0.2]$ percentage points for Kimi and $[-8.7, -0.3]$ for GPT. The fallback-demand contrasts with TTL-1d also remain negative ($[-87.9, -66.3]$ and $[-87.1, -67.1]$ points). These are comparisons within the restricted cache populations, not full-stream evaluations including the removed entries.

This replay removes the need to assume unchanged routing when describing the restricted caches. It does not establish policy-specific fallback quality, quantify the coverage cost of filtering, or make every transported end-to-end budget conservative. Tables S9.2 and S9.4 retain the original routing inputs and should be interpreted accordingly.

Table S9.2: Expected erroneous deliveries and abstentions per 1,000 Kimi answer–tick cases under per-part acceptance ($a = 11/20$, $ar = 0.15$), transported from the executed routing counts. The cached-answer error b is the clean-stratum point estimate (0) or its Wilson upper bound (16.1%). Abstentions do not depend on b. Transport assumptions are stated in the text.

		$b = 0$		$b = 16.1\%$
policy	π	errors	abstain	errors
similarity-only	–	42.7	0.0	196.9
KV-claims	1.000	6.4	19.2	160.5
sent-asserted	0.225	28.8	83.8	159.6
page-hash	0.053	121.8	365.4	152.1
TTL-1d	0.045	142.5	427.5	150.6

Table S9.3: Expected erroneous deliveries per 1,000 GPT answer–tick cases under set-level acceptance ($b = 22.8\%$, $ar = 0.50$, $a = 19/20$); companion to main-text Table 5.

policy	π	stale	cached error	fallback error	total	abstain
similarity-only	–	39.0	219.1	0.0	258.1	0.0
KV-claims	1.000	0.0	219.1	19.5	238.6	1.9
sent-asserted	0.194	0.7	182.9	98.5	282.1	9.9
page-hash	0.048	0.0	41.4	409.2	450.6	40.9
TTL-1d	0.041	0.0	11.4	475.0	486.4	47.5

Verdict probabilities. We use independent Jeffreys posteriors for erroneous fallback delivery mass ar, based on 10/20 set-level or 3/20 per-part outcomes. For set-level cached error, b is the mixture of flagged (18/20) and clean (0/20) strata with fixed weights 33/130 and 97/130. The per-part analysis uses the clean stratum alone. Table S9.4 uses 10^6 draws with seed 42.

For the Kimi asserted gate, improvement over TTL-1d requires $ar - b > (1 - b)/947$. The probability agrees with $\Pr(ar > b)$ to two decimals. KV-claims, which rejects exactly the stale cases, is error-optimal within the homogeneous reuse-or-fallback model when $ar \geq b$ (main-text Corollary 5.1(i),(iii)). The equality event has zero posterior probability under these continuous models.

These are conditional sensitivity summaries, not probabilities of deployment superiority. They assume fixed routing inputs, fixed stratum weights, independent posteriors, and transport of sampled quality across policies and ticks. The per-part row retains the original routing precision despite changed cache eligibility. The calculation does not account for violations of these modeling assumptions or variability across repeated generation runs.

Table S9.4: Conditional posterior summaries of the projected end-to-end comparisons for the Kimi asserted gate (independent Jeffreys posteriors; 10^6 Monte Carlo draws, seed 42). The last column also gives the probability that KV-claims is error-optimal. Per-part rows transport the executed routing counts.

acceptance	beats similarity-only	beats TTL-1d	$ar > b$ (KV optimal)
set-level	0.22	0.99	0.99
per-part	0.83	0.97	0.97

S10 Experiment Inventory and Artifact Manifest

S10.1 Experiment inventory and data sources

Table S10.1 indexes the reported experiments, analytical references, descriptive analyses, and implementation checks. It identifies their data sources, retained result files, and relevant sections. Script-level execution details are given in the corresponding protocols and artifact documentation.

Artifact status. The filenames in this inventory identify the study's retained inputs, scripts, and outputs; they do not by themselves establish independent reproducibility. The artifacts of experiments 01–16 (controlled audits, snapshot dynamics, the monthly serving replay, the daily-slice replay, and the small-schema enumeration check) are publicly available at `https://github.com/xuhu6736/proof-valid-benchmarking`. Monthly-replay files reside under `experiments/14_snapshot_serving_replay/` (`data/`, `scripts/`, `annotation/`); daily-slice files under `experiments/15_second_domain_daily_replay/`.

Result sources. Primary endpoint counts use the label-aware fields in `real_driven_loop.json`. Primary multi-tick annotation agreement uses `annotation/tableb_agreement.json`. Replacement-generator endpoint counts use `gpt_reexec_endpoint.json`. Fault routing and p95 values use `real_driven_sweep.json`; label-aware fault quality uses `real_driven_loop.json`.

The earlier `tableb_endpoint_map.json` and the per-arm counts in `tableb_endpoint_quality.json` do not define the reported primary routing. Daily quality projections use item labels, acceptance decisions, and routing counts. Legacy confidence intervals and embedded window verdicts in `base_answer_results.json` are not used. The manuscript reports the stratified sensitivity analysis in Section S9.2. Trace operating points use the SearchQueries records in `trace_endpoints_sq_results.json` and the LmArena records in `trace_endpoints_results.json`. The SearchQueries $q = 0.90$ records in the latter file are retained references, not the reported operating points. Daily resampling intervals use the multiplicity-preserving paired page-cluster implementations in `stage13_pagecluster_bootstrap.py` and `stage14_perpart_filtered_replay.py`. The envelope audit and payload-chain component timings come from separate earlier mechanism executions, not the headline real-output trajectories.

Table S10.1: Experiment inventory. HP = HotpotQA; MQ = MuSiQue; SQ/LA = SearchQueries/LmArena; TW = TemporalWiki; EN/WD = English Wikipedia/Wikidata; WFD = WikiFactDiff. Artifact names identify retained result files.

Experiment	Data	Output artifact	Section
Grounded audit + stress	HP + SQ/LA	results_grounded.json, main-text Figure 3	§S1.1
Footprint count perturbation	HP	e13_constructor_noise_results.json, e13_unbalanced_sensitivity.json	§S1.9
Coded storage references	HP	coded_regime_results.json, Figure S2.1	§S2.1
Heuristic support families	HP	e11_or_strict_results.json, e11_exact_family_agreement.json, Figure S2.4	§S2.3
Shared-root storage audit	HP, MQ	module_selection_results.json, e12_module_musique_results.json, Figure S2.2	§S2.1
Equal-budget gain references	HP, MQ	e6b_natural_context_results.json, Figure S2.3	§S2.2
Grounded operating points	HP, MQ	grounded_operating_results.json, Table S2.1	§S2.1
Instrumented retrieval audit	HP (66,581 docs)	e8_pipeline_results.json, Figure S1.1	§S1.2
Local timing microbenchmark	HP dataset-context surrogates	remeasure_timing_m3pro.json, remeasure_gate_intact_m3pro.json, e8_timings_m3pro.npz	§S1.2
Correlated erasure	HP, MQ	e9_correlated_results_matched.json, main-text Figure 4	§S1.2
Error–throughput curves	HP	e10_pareto_results.json, Figure S1.2	§S1.8
Trace operating points	SQ, LA	trace_endpoints_results.json, trace_endpoints_sq_results.json, Table S1.2	§S1.6
Snapshot change rates	EN 5 snap. + WD 4 snap. 2026	eps_t.json, eps_stmt_v2.json, Table S3.1 and Figure S3.1	§S3.1
Replacement rates + popularity	WFD	wfd_popularity_workload.json, workload_corrected_2026.json, Figure S3.1(b)	§S3.1
Snapshot cache construction and screening	EN/WD panel, generations, embeddings	replay_summary.json, replay.json, unified_retrieval.json, tau_oos.json, cluster_bootstrap.json	Main text
One-shot quality and regeneration	Same panel, human labels	regeneration.json, closed_loop.json, unified_strategy_table.json, regen_labels.json, `annotation/` sheets + labels	§S4
Exploratory alternatives	Same serving panel	alternative_certificates.json, qualified_alternatives.json, simple_baselines.json, llm_judge.json	§S5
Matched endpoint quality	Archived generations and human labels	real_driven_loop.json, tableb_labels.json, regen_labels.json, annotation/tableb_agreement.json	Main text
Second-generator re-execution	Archived `kimi-k2-turbo-preview` records, `gpt-5-mini` re-run, trajectory re-execution, new human labels	tableb_regeneration_gpt.json, regeneration_gpt.json, gpt_generator_compare.json, gpt_labels.json, gpt_tableb_endpoint_quality.json, gpt_maintenance_counts.json, gpt_reexec_endpoint.json, annotation/gpt_agreement.json, annotation/pairing_audit.json	§S4.4

(continued on next page)

Table S10.1: Experiment inventory (continued)

Experiment	Data	Output artifact	Section
Exploratory gate mappings, claim-equality ablation, and crossed gate-by-alias control	Same serving panel, archived real generations	mechanism_arms.json, claim_equality_loop.json, claim_equality_loop_alias.json, gate_alias_cross.json, gate_alias_request_diff.json, Tables S4.7, S4.8	§S4.5
Real-driven fault sweep	Local payload chain, injected reads	tableb_regeneration.json, real_driven_loop.json, real_driven_sweep.json	Main text
Budgeted protection and restore diagnostics	Payload store, fault seeds	protection_recovery.json, fixed_budget_protection.json, protected_closed_loop.json, sensitivity_ttl_vs_loop.json, closed_loop_system.json, `current_claims/`	§S6, §S7
Residual and paired attribution	Paired fault trajectories	residual_decomposition.json, paired_transitions_real.json (headline); paired_transition.json (earlier mechanism check)	§S7
Frozen new-object check	Non-design entities	frozen_panel_generation.json, frozen_panel_check.json	Main text
Certificate handoff	Transferred certificate envelopes	certificate_handoff.json	§S7.2
Payload-chain component microbenchmark	Earlier local-store mechanism replay	real_payload_chain.json (component timings and storage fields only)	§S7
Staged timing and fail-closed checks	Snapshot serving panel	staged_timing_failclosed.json	§S7.6
Retrieval scaling	Synthetic cache sizes	retrieval_scaling.json	§S7.6
Daily-slice replay (main scenario)	EN 99 pages, 21 daily ticks 2026-07, dual generators	series21.json, panel21.json, pool_stats21.json, replay_results21.json, dual-generator replay outputs, pagecluster_bootstrap.json (`stage13_pagecluster_bootstrap.py`), perpart_filtered_replay.json (`stage14_perpart_filtered_replay.py`), main-text Tables 3–4	Main text, §S9
Daily service-demand scenario	Gate timings, API and page-fetch probes	load_cost.json (`stage6_load_cost.py`), Table S8.1	§S8
Daily quality calibration and sensitivity	Dual-generator base answers and first-rejection fallbacks	fallback_regen.json, fallback_spotcheck_results.json, base_answer_sample40.json, base_answer_results.json, perpart_rescore.json, `annotation/` sheets	§S9.2
Bootstrap denominator audit	Daily full and per-part-filtered caches	zero_denominator_check.json (`stage15_zero_denominator_check.py`)	§S9.1
Small-schema state-enumeration check	Generated conjunctive DAGs (400)	dag_correctness_results.json (`experiments/16_dag_correctness_test/`)	§S1.10

References

[1] Bang, F.: GPTCache: an open-source semantic cache for LLM applications enabling faster answers and cost savings. In: Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS 2023), pp. 212–218 (2023)

[2] Schroeder, L.G., Desai, A., Cuadron, A., *et al.*: vCache: verified semantic prompt caching. In: International Conference on Learning Representations (2026)

[3] Yang, Z.L., Qi, P., Zhang, S.Z., *et al.*: HotpotQA: a dataset for diverse, explainable multi-hop question answering. In: Proceedings of the Conference on Empirical Methods in Natural Language Processing, pp. 2369–2380 (2018)

[4] Trivedi, H., Balasubramanian, N., Khot, T., *et al.*: MuSiQue: multihop questions via single-hop question composition. Trans Assoc Comput Linguist **10**, 539–554 (2022)

[5] Reimers, N., Gurevych, I.: Sentence-BERT: sentence embeddings using Siamese BERT-networks. In: Proceedings of the Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, pp. 3982–3992 (2019)

[6] vCache Project: SemCache benchmark datasets and evaluation code. GitHub Repository. `https://github.com/vcache-project/vCache`. Accessed August 2026 (2026)

[7] Xu, J.: Partial path interception in derivation DAGs: exact reliability laws and a sharp complexity dichotomy. Version 3; earlier versions appeared under the title "Proof-Valid Caching under Premise Erasures" (2026)

[8] Wikimedia Foundation: Wikimedia database backup dumps: enwiki and wikidatawiki snapshots, 2026-04-01 to 2026-08-01. `https://dumps.wikimedia.org/`. Accessed August 2026 (2026)

[9] Jang, J., Ye, S., Lee, C., *et al.*: TemporalWiki: a lifelong benchmark for training and evaluating ever-evolving language models. In: Proceedings of the Conference on Empirical Methods in Natural Language Processing, pp. 6237–6250 (2022)

[10] Ammar Khodja, H., Béchet, F., Brabant, Q., *et al.*: WikiFactDiff: a large, realistic, and temporally adaptable dataset for atomic factual knowledge update in causal language models. In: Proceedings of the Joint International Conference on Computational Linguistics, Language Resources and Evaluation, pp. 17614–17624 (2024)

[11] Yoon, Y., et al.: Risk-constrained freshness-aware semantic caching for open-web retrieval-augmented LLMs. arXiv:2607.04281v1 (2026)

[12] Shah, S.H.: Grounded cache routing for retrieval-augmented generation: when is it safe to reuse an answer? arXiv:2605.27494v1 (2026)

[13] Zeng, L., Jin, Y.: FinCacheServe: dependency-consistent answer reuse for cost-efficient RAG serving over mutable enterprise documents. arXiv:2607.26076v1 (2026)